



DSP Integrated Circuits

Chapter 3: DFT & FFT

Prof. Jaeseok Kim

Communication SoC Design Lab.

Yonsei University

Contents

- ◇ DFT (Discrete Fourier Transform)
- ◇ FFT (Fast Fourier Transform)
 - Radix-2 DIT (Decimation In Time)
 - Radix-2 DIF (Decimation In Frequency)
 - Radix-r & Composite N
- ◇ Windows & Window weighting
- ◇ H/W design of FFT block
 - Multiplier & Butterfly architecture
 - FFT architecture

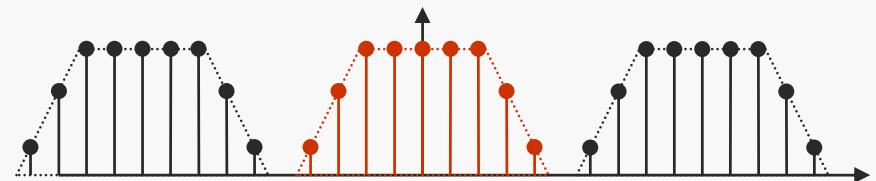
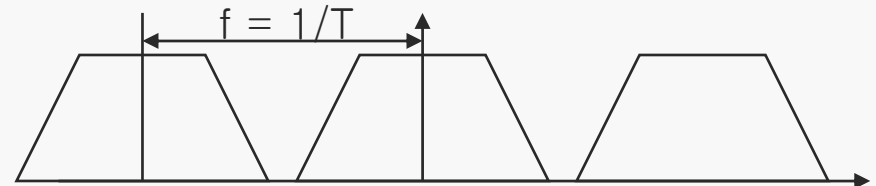
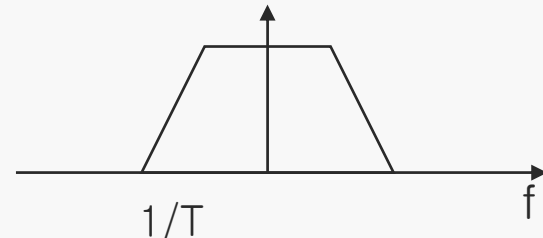
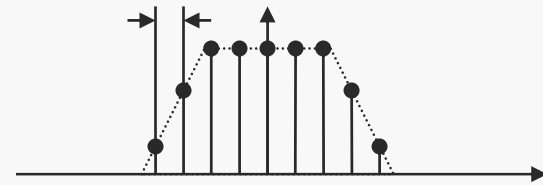
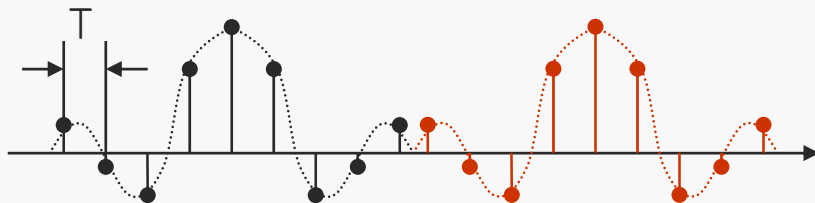
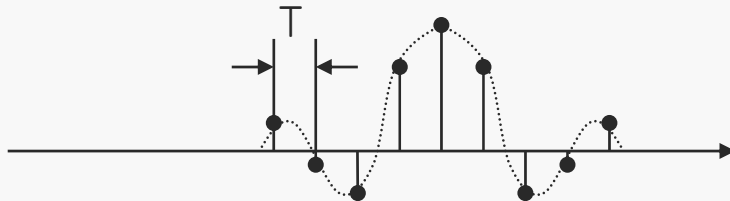
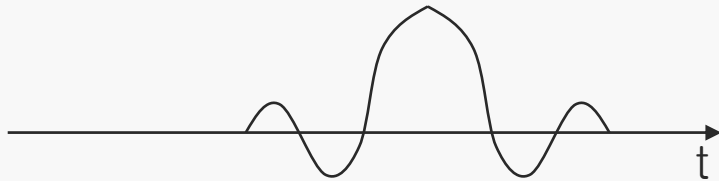
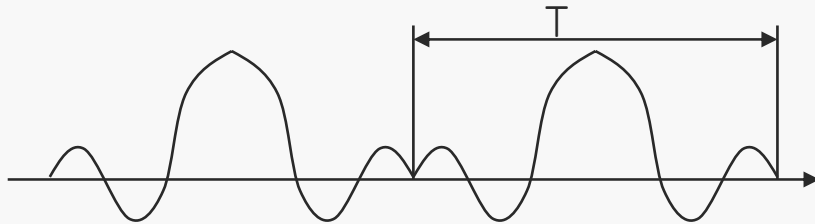


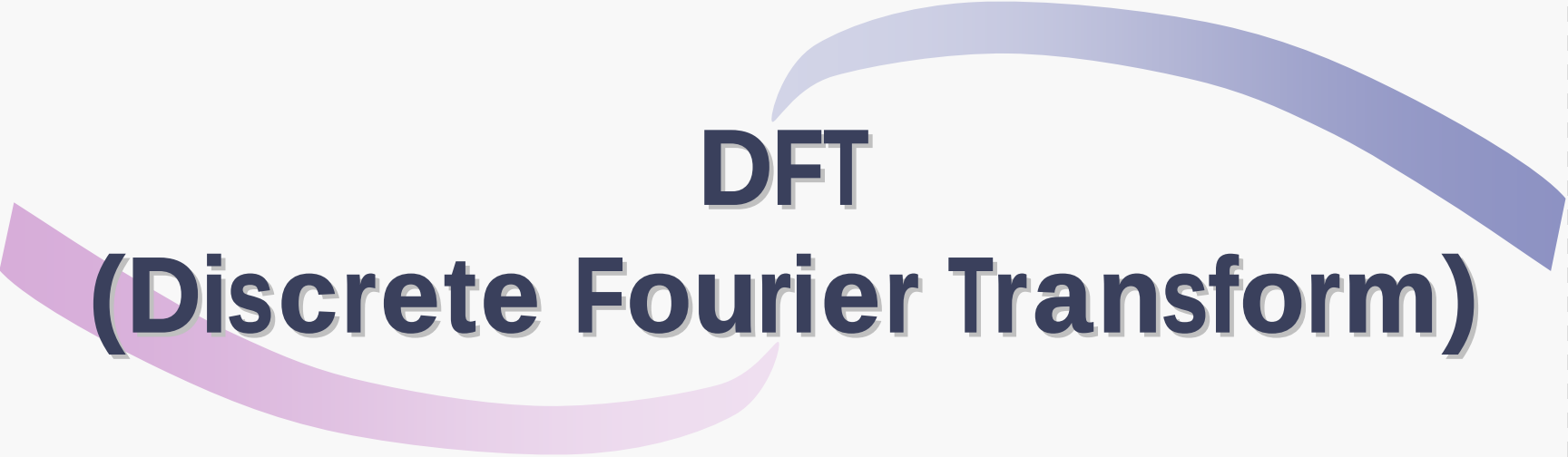
Summary of Fourier Analysis

	Time domain $y(t)$	Frequency domain $Y(f)$
Fourier series	Continuous Periodic	Discrete Infinite
Fourier Transform	Continuous Aperiodic	Continuous Infinite
FT of sampled data	Discrete (sampled) Aperiodic	Continuous Periodic
DFT (Discrete Fourier Transform)	Discrete Periodic	Discrete Periodic



Pictorial Representation of F.A.





DFT

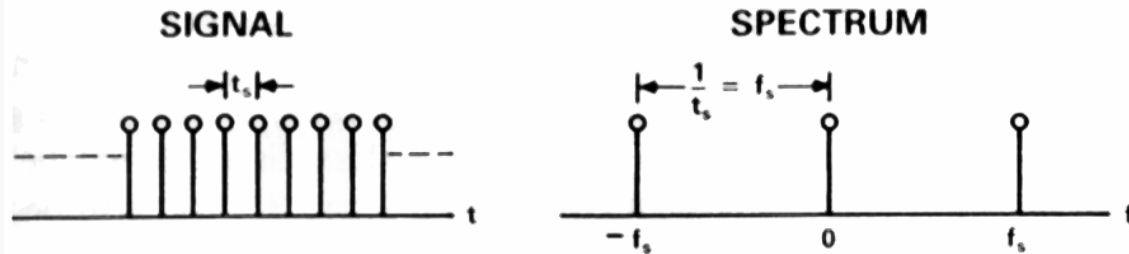
(Discrete Fourier Transform)

DFT (Discrete Fourier Transform)

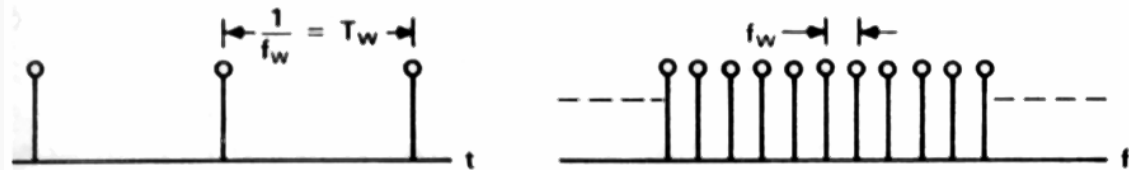
- ◇ DFT: Fourier transform of a sampled signal computed at equally spaced frequency intervals of size f_s/N , where N is # of points in sampled window
- ◇ DFT special properties
 - Inverse DFT gives a reconstructed signal which is periodic in interval at the window, $T_w = N \cdot t_s$
 - DFT gives as many points in spectrum as there are in sampled signal
 - DFT is a complete representation of sampled signal



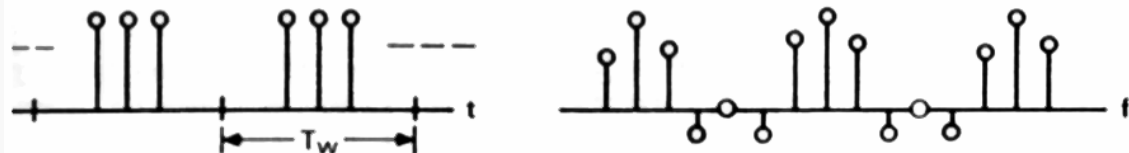
Periodicity of DFT



a. Sampling in time generates periodicity in frequency.



b. Sampling in frequency generates periodicity in time.



c. Sampled FT or DFT for a pulse.

DFT/FFT Applications

Signal processing

- matched filters
- real time spectral analysis
- coherence function estimation

Communications

- OFDM
- demodulators
- multichannel filtering
- M-ary signaling
- voice coding system

Sonics and Acoustics

- acoustic imaging
- passive sonar
- music synthesis

Radar

- cross-section measurement
- moving target indicator
- doppler processor

Biomedical Engineering

- diagnosis of airways obstruction
- muscle fatigue monitoring

Instrumentation

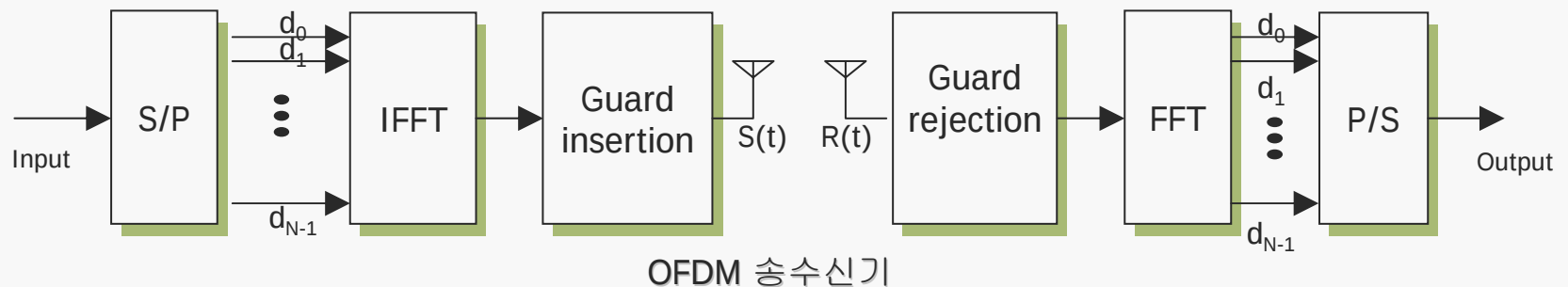
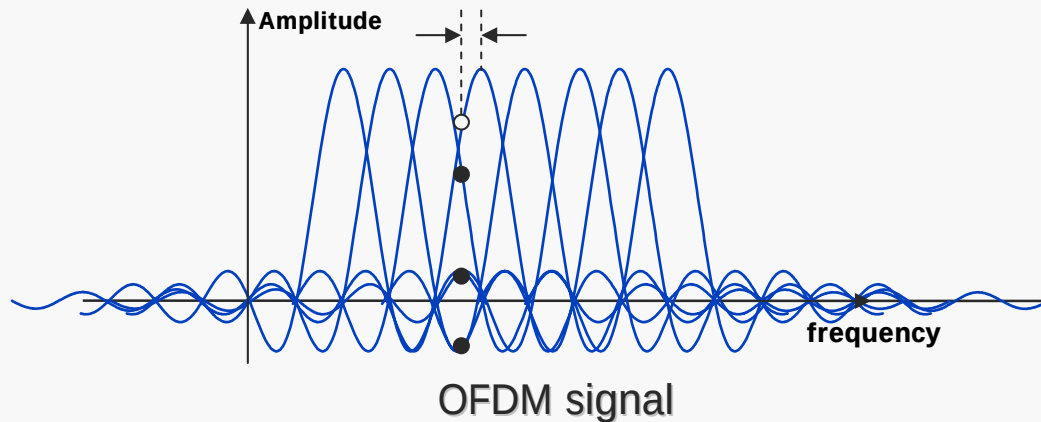
- chromatography
- microscopy
- x-ray diffraction



FFT Application: OFDM

◇ OFDM (Orthogonal Frequency Division Multiplexing)

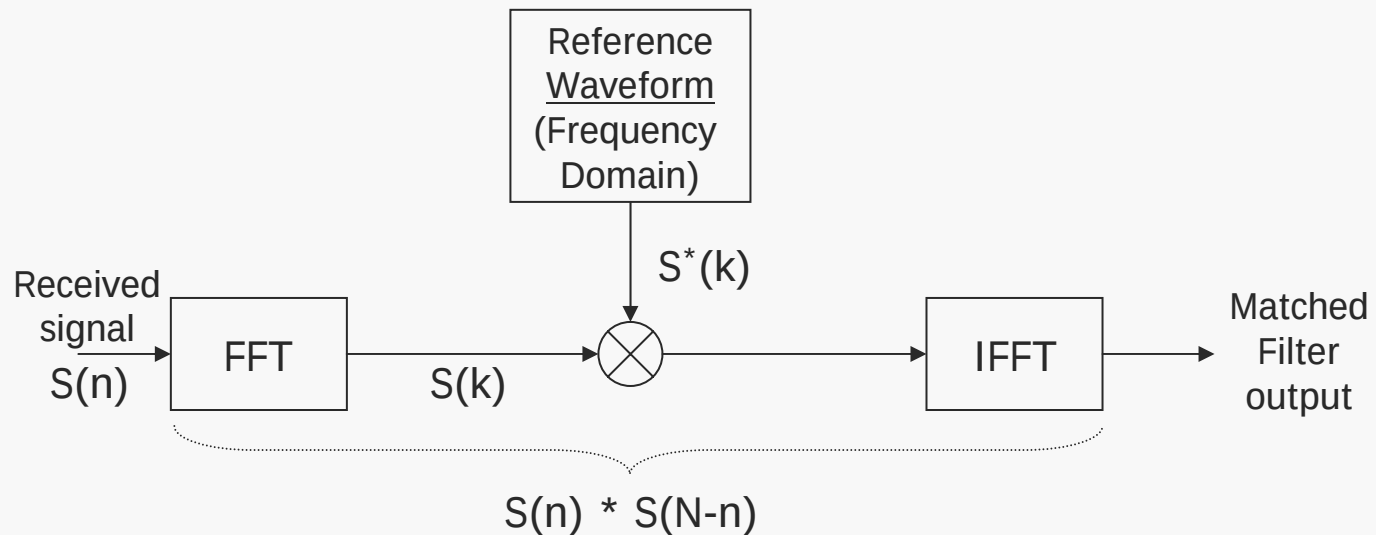
- 주파수 영역에서 직교성을 가지는 subcarrier들로 정보를 전송하는 방식



◇ FFT/IFFT를 이용하여 Multicarrier 구현 (DAB, ADSL, WLAN 등에 이용)

FFT Application: Matched Filter

- FFT/IFFT를 이용하여 Time domain 신호의 convolution을 Frequency domain 신호의 곱셈으로 구현



DFT Definition

- ◆ The discrete Fourier transform (DFT) of discrete signal $x(n)$ is defined as

$$X(k) \triangleq \sum_{n=0}^{N-1} x(n) W_N^{nk} \quad , \quad k = 0, 1, \dots, N-1$$

and the inverse discrete Fourier transform (IDFT) is

$$x(n) = \frac{1}{N} \sum_{k=0}^{N-1} X(k) W_N^{-nk} \quad , \quad n = 0, 1, \dots, N-1$$

where $W_N = e^{-j2\pi/N}$

and $x(n)$ & $X(k)$ are complex sequences of length N

- ◆ Direct Computation of DFT/IDFT requires N^2 complex arithmetic operations (N^2 complex multiplications & $N(N-1)$ additions)



DFT Computation

Discrete Fourier Transform의 general form

$$X[k] = \sum_{n=0}^{N-1} x[n] W_N^{nk}, \quad k = 0, 1, 2, \dots, N-1 \text{ where } W_N = e^{-j(2\pi / N)}$$

$$= \sum_{n=0}^{N-1} [(\operatorname{Re}\{x[n]\} \operatorname{Re}\{W_N^{nk}\} - \operatorname{Im}\{x[n]\} \operatorname{Im}\{W_N^{nk}\}) + j(\operatorname{Re}\{x[n]\} \operatorname{Im}\{W_N^{nk}\} + \operatorname{Im}\{x[n]\} \operatorname{Re}\{W_N^{nk}\})],$$

$$k = 0, 1, 2, \dots, N-1$$

DFT computation

	Multiply	Add
Real	$4N^2$	$N(4N-2)$
Complex	N^2	$N(N-1)$

Example of DFT computation (Real number)

Features System	# of points (N)	Symbol rate (Required DFT time)	# of real multiplication	Required MAC time
ADSL	256	250μs	262,144	0.9ns
COFDM(DAB)	8192	1ms	268,435,456	3.7ps





FFT (Fast Fourier Transform)

- Radix-2 DIT (Decimation In Time)
- Radix-2 DIF (Decimation In Frequency)
- Radix-r & Composite N

FFT (Fast Fourier Transform)

- ◇ In 1965, Cooley and Turkey developed a fast algorithm based on the divide-and-conquer principle which takes an N -point transformation and split it into two $N/2$ -point transforms
 - Repeat until there remain $(N/2)$ two-point transforms
 - $\log_2 N$ splits $\rightarrow (N/2) \cdot \log_2 N$ basic operations

- ◇ Classes of FFT algorithms
 - Radix-2 Decimation-in-Time (DIT) Algorithm
 - Radix-2 Decimation-in-Frequency (DIF) Algorithm
 - Radix- r FFT Algorithm
 - Algorithm for composite N



Radix-2 DIT Algorithm (1)

- ◇ $N=2^v$ 인 DFT의 Time domain에서의 Decimation

$$X[k] = \sum_{n=0}^{N-1} x[n] W_N^{nk}, \quad k = 0, 1, \dots, N-1$$

- Separating $x[n]$ into its even- and odd-numbered points

$$X[k] = \sum_{n \text{ even}} x[n] W_N^{nk} + \sum_{n \text{ odd}} x[n] W_N^{nk}$$

- With the substitution of variables $n = 2r$ for n even and $n = 2r + 1$ for n odd

$$\begin{aligned} X[k] &= \sum_{r=0}^{(N/2)-1} x[2r] W_N^{2rk} + \sum_{r=0}^{(N/2)-1} x[2r+1] W_N^{(2r+1)k} \\ &= \sum_{r=0}^{(N/2)-1} x[2r] (W_N^2)^{rk} + W_N^k \sum_{r=0}^{(N/2)-1} x[2r+1] (W_N^2)^{rk} \end{aligned}$$



Radix-2 DIT Algorithm (2)

- By using $W_N^2 = W_{N/2}$ Characteristics

$$W_N^2 = e^{-2j(2\pi/N)} = e^{-j2\pi/(N/2)} = W_{N/2}$$

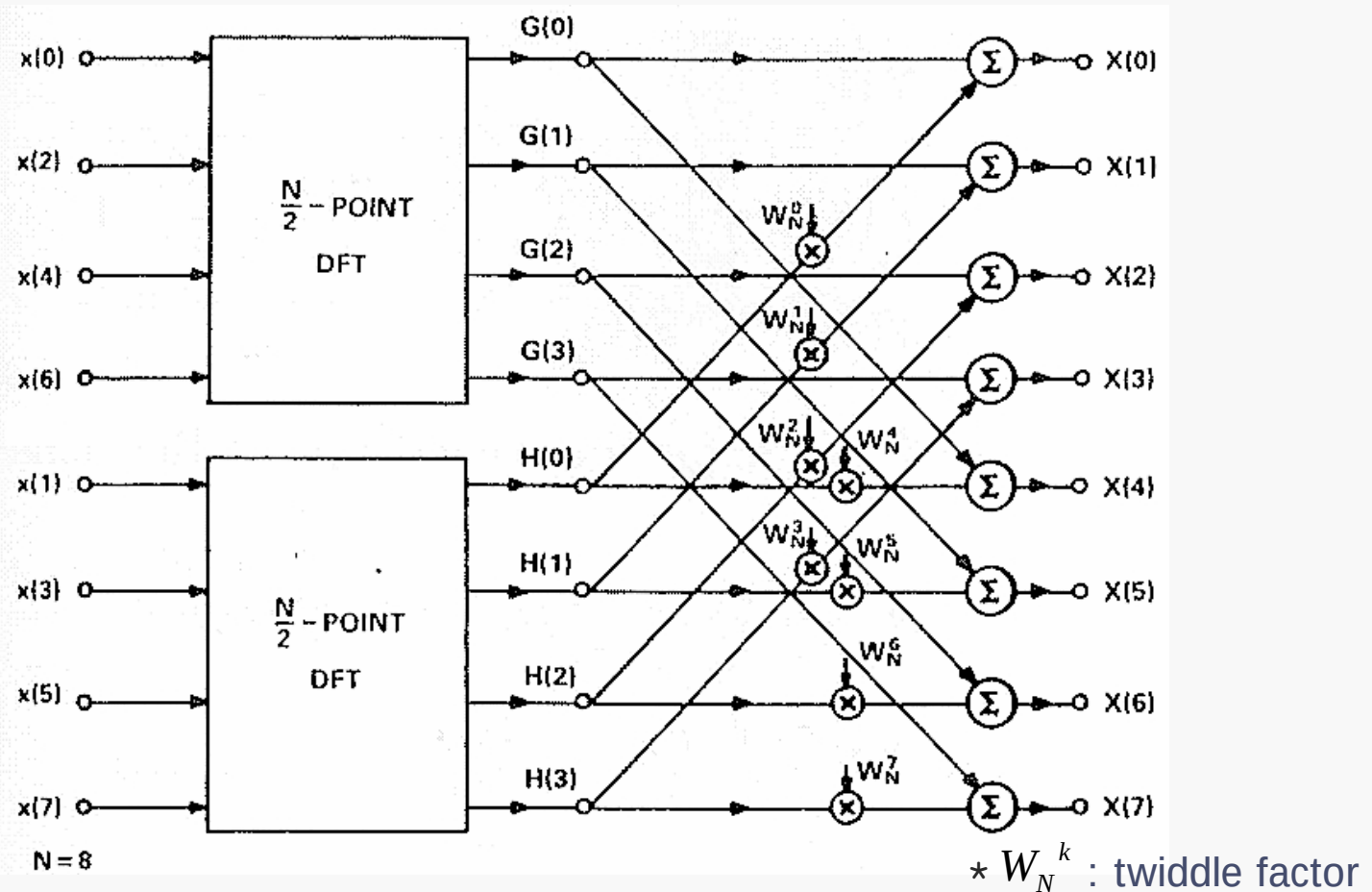
- $$\begin{aligned} X[k] &= \sum_{r=0}^{(N/2)-1} x[2r]W_{N/2}^{rk} + W_N^k \sum_{r=0}^{(N/2)-1} x[2r+1]W_{N/2}^{rk} \\ &= G[k] + W_N^k H[k] \end{aligned}$$

- * $G[k]$: the 4-point DFT of the even-numbered points
- * $H[k]$: the 4-point DFT of the odd-numbered points



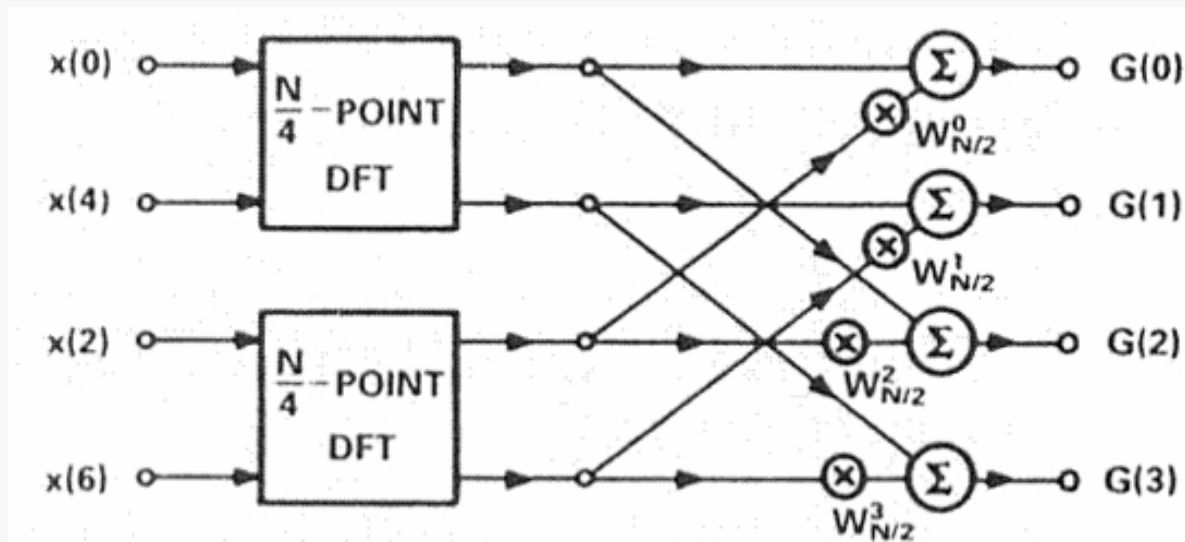
Radix-2 DIT Algorithm (3)

◇ Corresponding flow graph



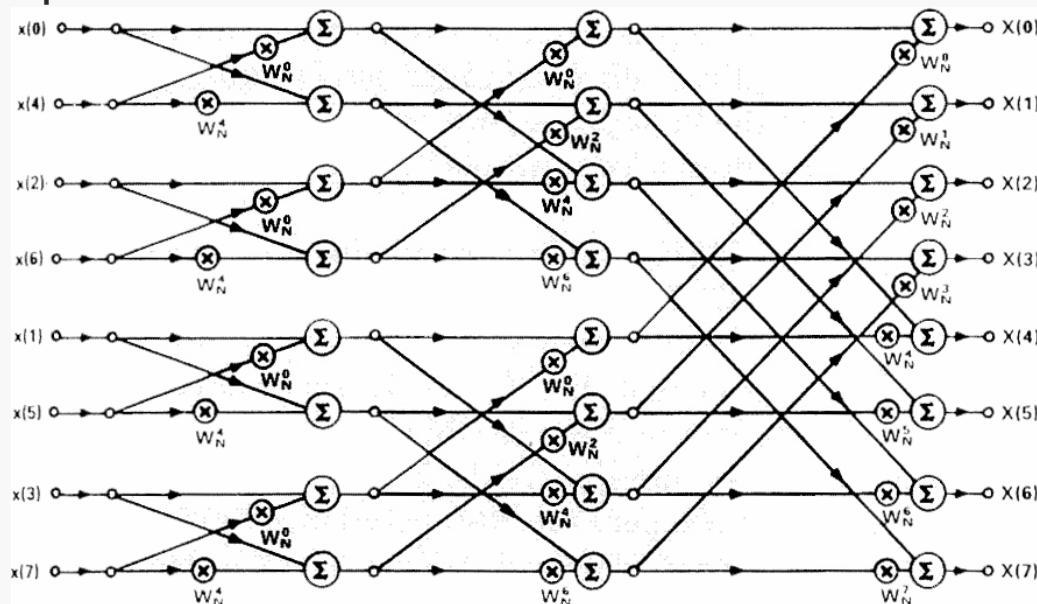
Radix-2 DIT Algorithm (4)

- Splitting 4-point transform into 2-point transform



Radix-2 DIT Algorithm (5)

- 동일한 방식으로 First term, Second term에 대해 각각 $N/4$, $N/8$, $\dots$, 2-point DFT로 연속해서 분리
- Final Flow graph



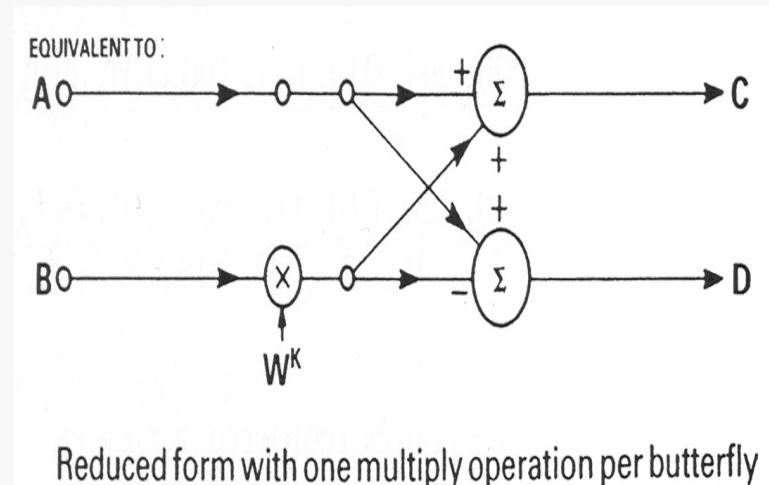
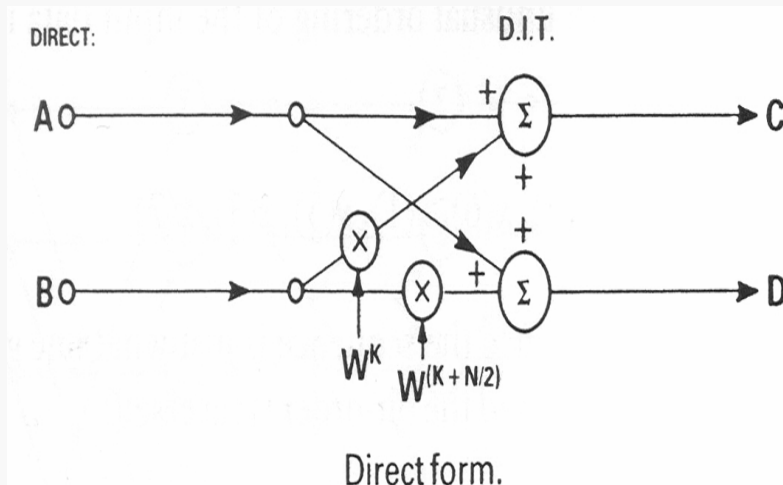
- Main computation: Butterfly structure
- Flow graph input sequence is bit-reversed order, output sequence is normal order

Butterfly Structure

- ◇ Butterfly (BF) structure
- ◇ Combines two input (A, B) to give two outputs(C, D) as

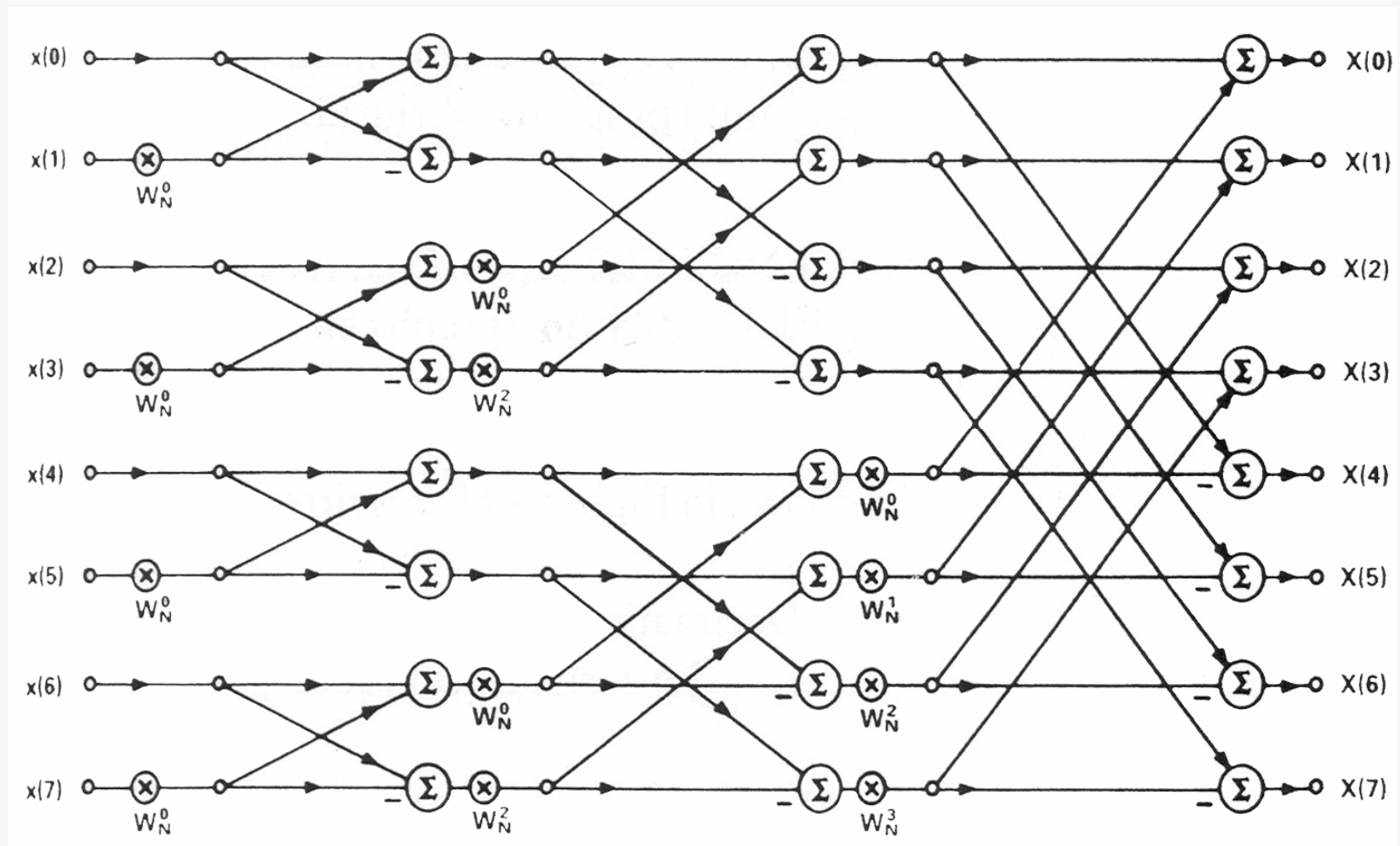
$$C = A + w^k B$$

$$D = A - w^k B$$



- Mth stage의 output sequence는 Butterfly 연산을 수행한 후 input sequence의 memory에 저장할 수 있음(*In-place algorithm*)

Signal-flow graph for the 8-point DIT



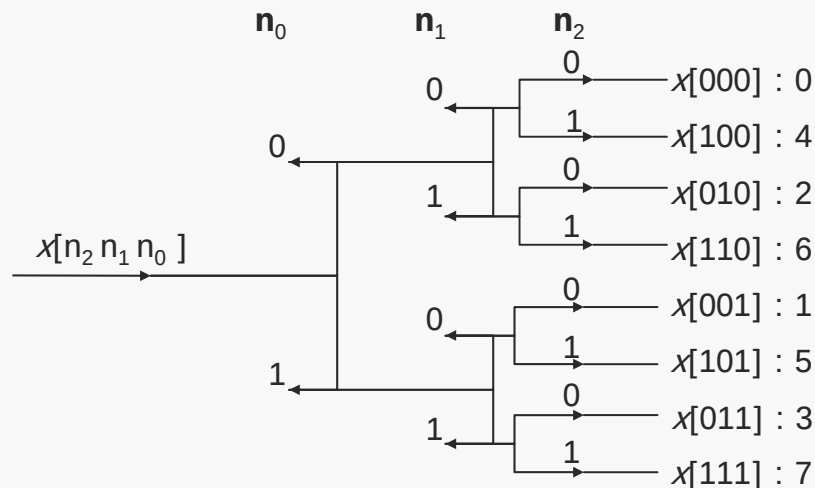
Bit-reversed order

◇ Bit-reversed order

■ DIT(DIF)방식에서 *In-Place* 알고리즘을 사용하기 위해서는

- output sequence is normal order
→ access input sequence in bit-reversed order
- input sequence is normal order
→ access output sequence in bit-reversed order

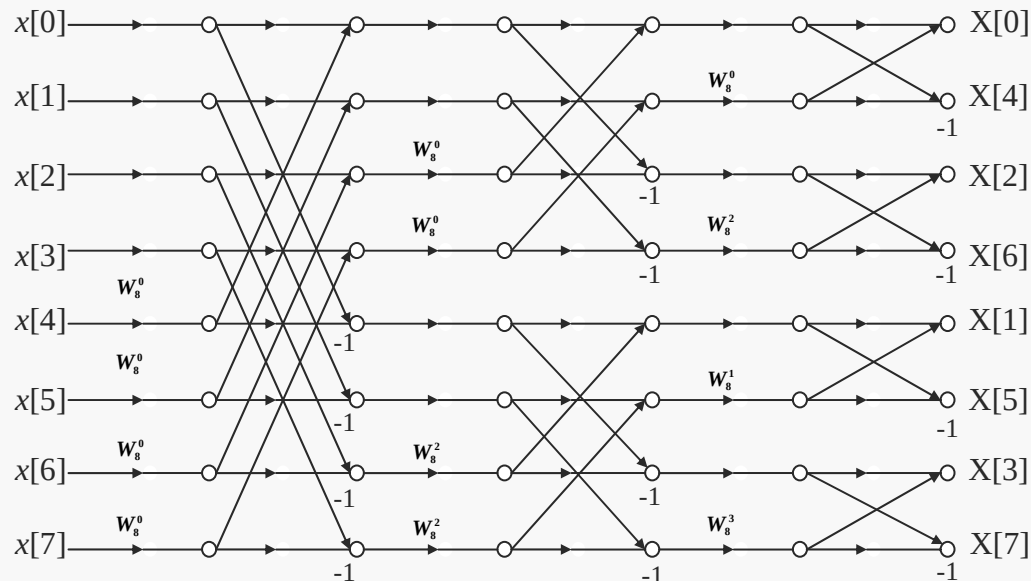
Ex) 8-point DIT input sequence의 bit-reversed order



Alternative Forms

◇ Alternative Forms in DIT

- Input sequence is normal order, Output sequence is bit-reverse order



Comparison of FFT and DFT

◇ Computation of DFT/FFT for ADSL & DAB

	# of Point	Symbol rate	# of DFT Computation	# of FFT Computation
ADSL	256	250 μ s	262,133	4,032
COFDM (DAB)	8,196	1 ms	268,435,456	213,096

◇ Required MAC time

	Required MAC time(DFT)	Required MAC time(FFT)
ADSL	0.9 ns	62 ns
COFDM (DAB)	3.7 ps	4.7 ns



Radix-2 DIF Algorithm (1)

- ◇ $N=2^v$ 인 DFT의 frequency domain에서의 decimation

$$X[k] = \sum_{n=0}^{N-1} x[n] W_N^{nk}, \quad k = 0, 1, \dots, N-1$$

- Even-numbered frequency samples are

$$\begin{aligned} X[2r] &= \sum_{n=0}^{N-1} x[n] W_N^{n(2r)}, \quad r = 0, 1, \dots, (N/2)-1, \\ &= \sum_{n=0}^{(N/2)-1} x[n] W_N^{2nr} + \sum_{n=N/2}^{N-1} x[n] W_N^{2nr} \\ &= \sum_{n=0}^{(N/2)-1} x[n] W_N^{2nr} + \sum_{n=0}^{(N/2)-1} x[n + (N/2)] W_N^{2r[n + (N/2)]} \end{aligned}$$

- Because of

$$\bullet W_N^{2r[n + (N/2)]} = W_N^{2rn} W_N^{rN} = W_N^{2rn}$$

$$\bullet W_N^2 = W_{N/2}$$

■
$$X[2r] = \sum_{n=0}^{(N/2)-1} (x[n] + x[n + (N/2)] W_{N/2}^{rn}), \quad r = 0, 1, \dots, (N/2)-1$$



Radix-2 DIF Algorithm (2)

- Odd-numbered frequency samples are

$$\begin{aligned} X[2r+1] &= \sum_{n=0}^{N-1} x[n] W_N^{n(2r+1)}, \quad r = 0, 1, \dots, (N/2) - 1 \\ &= \sum_{n=0}^{(N/2)-1} x[n] W_N^{n(2r+1)} + \sum_{n=N/2}^{N-1} x[n] W_N^{n(2r+1)} \\ &= \sum_{n=0}^{(N/2)-1} (x[n] - x[n + (N/2)]) W_N^{n(2r+1)} \end{aligned}$$

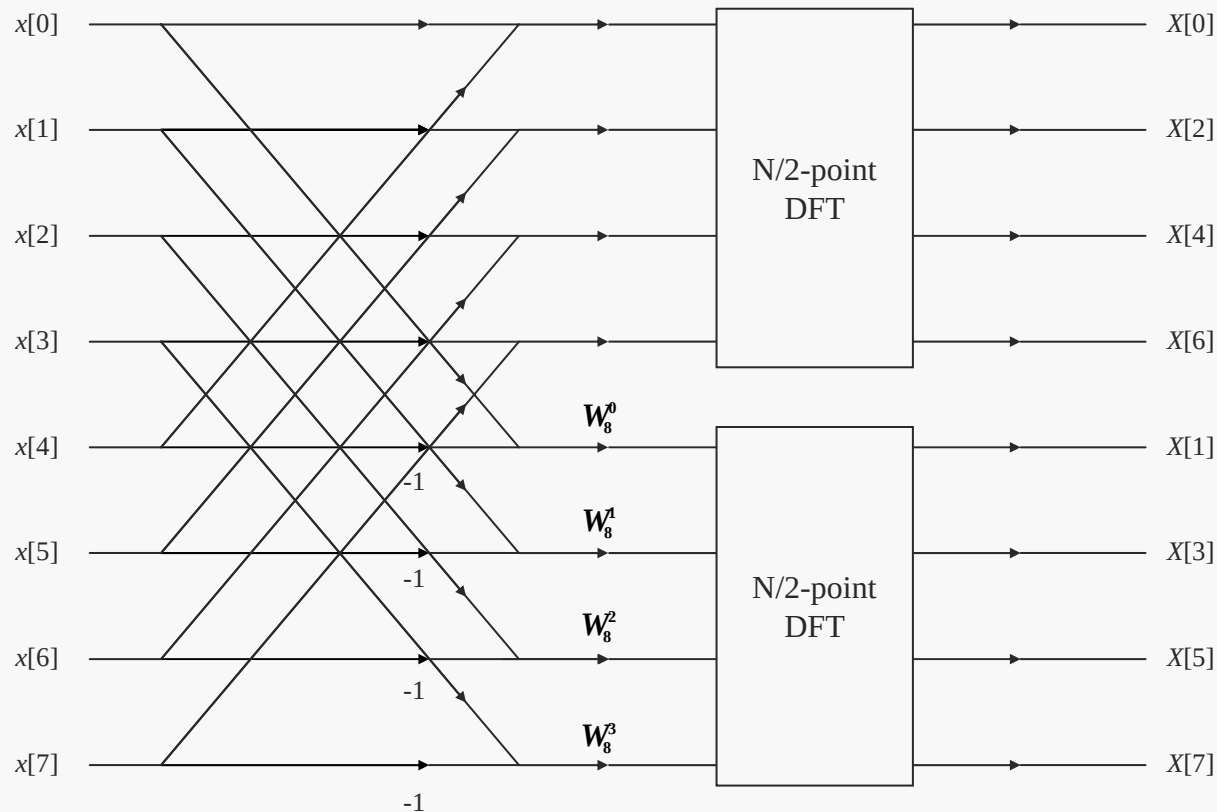
- Since $W_N^2 = W_{N/2}$,

$$X[2r+1] = \sum (x[n] - x[n + (N/2)]) W_N^n W_{N/2}^{nr}, \quad r = 0, 1, \dots, (N/2) - 1$$



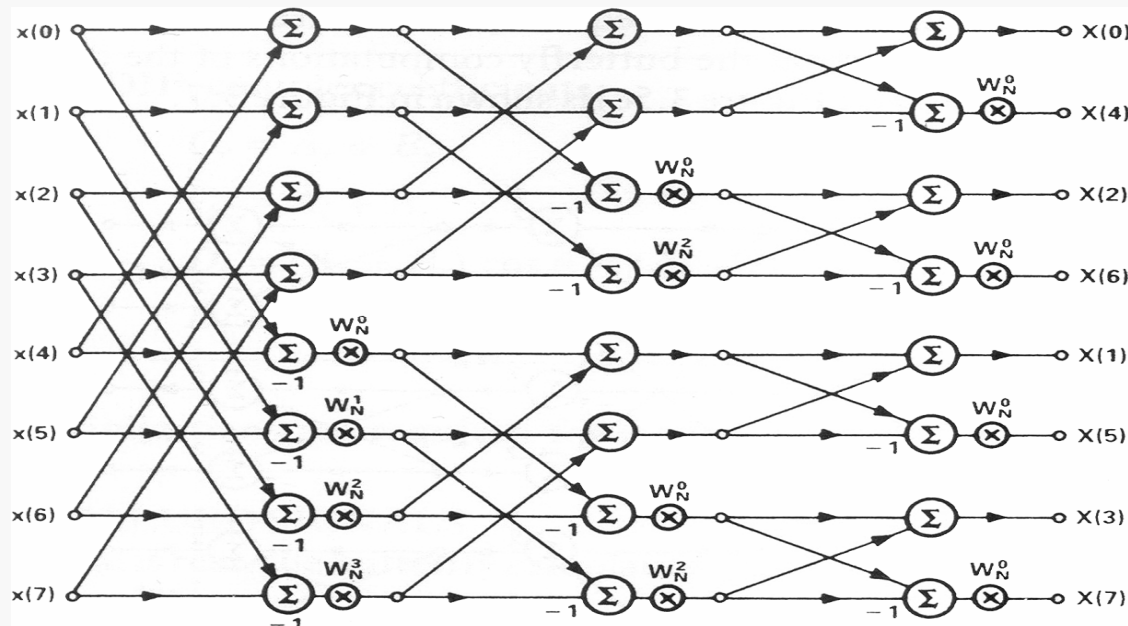
Radix-2 DIF Algorithm (3)

◇ Corresponding Flow graph



Radix-2 DIF Algorithm (4)

- ◆ DIT 알고리즘과 같은 방식으로 First term, Second term에 대해 각각 $N/4$, $N/8$, $\dots$, 2-point DFT로 연속해서 분리
- ◆ Final Flow graph



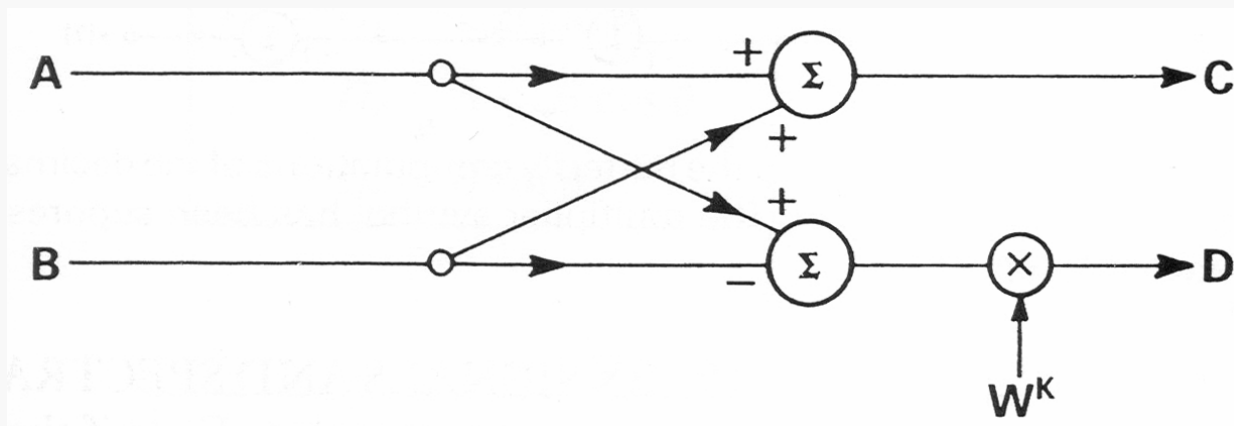
- ◆ Flow graph: input sequence is normal order, output sequence is bit-reversed order

Butterfly structure for DIF

- Combines two input (A, B) to give two outputs (C, D) as

$$C = A + B$$

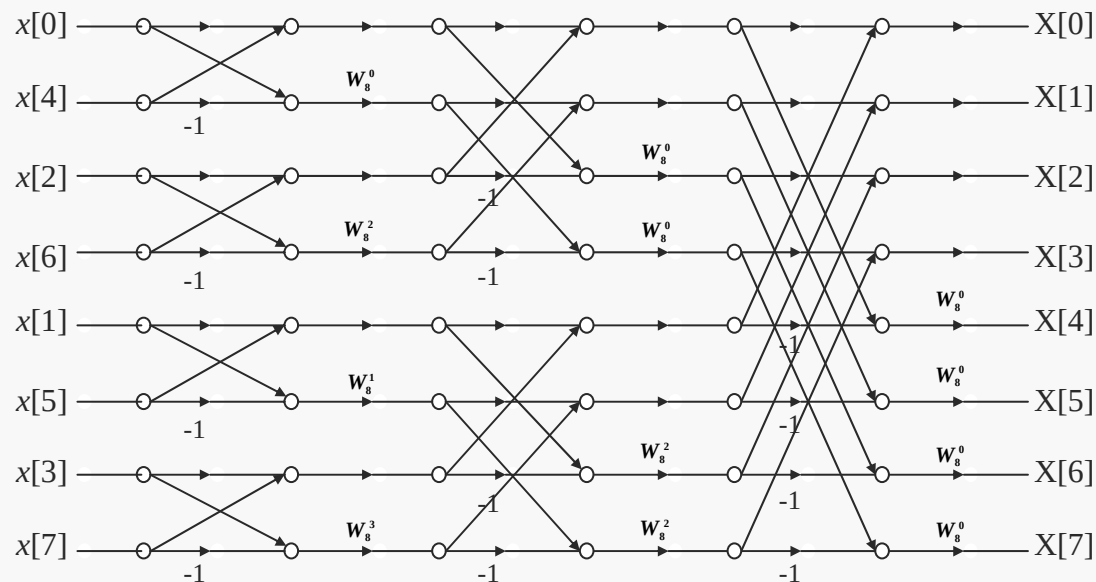
$$D = (A - B) w^k$$



Alternative Form

◇ Alternative Forms in DIF

- Input sequence is bit-reverse order가 되고, Output sequence is normal order

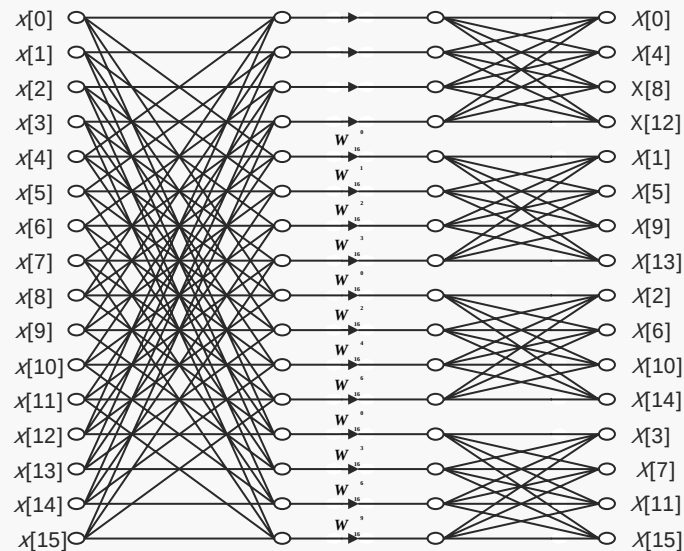
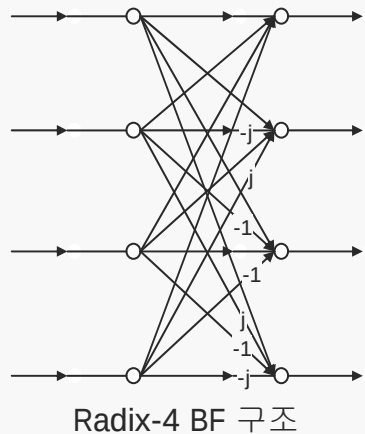


- 계산량은 DIT 방식과 동일함

Radix-r FFT

◇ Radix-r FFT method

- $N=R^v$ 인 경우, R개의 Input/Output을 갖는 BF 구조를 이용하여 DIF/DIT 방식으로 연산을 수행
- High order radix 알고리즘은 계산이 효율적이고, BF 개수와 stage는 줄여 주지만, BF 구조 자체는 복잡해짐
- $N=4^2$ 인 경우(Radix-4 방식)



Algorithm for composite N (1)

- ◇ $N=N_1 \times N_2$ 의 sequence length에 대한 일반적인 FFT 알고리즘
- ◇ Index mapping
 - mapping one-dimensional input sequence into $n_2 \times n_1$ two-dimensional array as following

$$\text{Input sequence :} \quad n = N_2 \cdot n_1 + n_2, \quad \begin{cases} 0 \leq n_1 \leq N_1 - 1 \\ 0 \leq n_2 \leq N_2 - 1 \end{cases}$$

$$\text{Output sequence :} \quad k = k_1 + N_1 \cdot k_2, \quad \begin{cases} 0 \leq k_1 \leq N_1 - 1 \\ 0 \leq k_2 \leq N_2 - 1 \end{cases}$$

◇ 유도

$$\begin{aligned} X[k] &= X[k_1 + N_1 k_2] \\ &= \sum_{n_2=0}^{N_2-1} \sum_{n_1=0}^{N_1-1} x[N_2 n_1 + n_2] W_N^{(k_1 + N_1 k_2)(N_2 n_1 + n_2)} \\ &= \sum_{n_2=0}^{N_2-1} \sum_{n_1=0}^{N_1-1} x[N_2 n_1 + n_2] W_N^{N_2 k_1 n_1} W_N^{k_1 n_2} W_N^{N_1 k_2 n_2} W_N^{N_1 N_2 k_2 n_1} \\ &= \sum_{n_2=0}^{N_2-1} \left[\left(\sum_{n_1=0}^{N_1-1} x[N_2 n_1 + n_2] W_{N_1}^{k_1 n_1} \right) W_N^{k_1 n_2} \right] W_{N_2}^{k_2 n_2} \dots \dots \dots \text{스!} * \end{aligned}$$

◇ 예 : $15=3 \times 15$



Algorithm for composite N (2)

◇ DFT Procedure

1) 5-point DFT of 3 rows

식 *의 내부 괄호 항 계산

$$G[n_2, k_1] = \sum_{n_1=0}^4 x[3n_1 + n_2] W_5^{k_1 n_1}, \quad \begin{cases} 0 \leq k_1 \leq 4 \\ 0 \leq n_2 \leq 2 \end{cases}$$

$k_1 \backslash n_2$	0	1	...	4
0	G[0,0]	G[0,1]	...	G[0, 4]
1	G[1,0]	G[1,1]	...	G[1, 4]
2	G[2,0]	G[2,1]	...	G[2,4]

2) Multiply twiddle factor

$$\tilde{G}[n_2, k_1] = W_{15}^{k_1 n_2} G[n_2, k_1], \quad \begin{cases} 0 \leq k_1 \leq 4 \\ 0 \leq n_2 \leq 2 \end{cases}$$

$k_1 \backslash n_2$	0	1	...	4
0	$\tilde{G}[0,0]$	$\tilde{G}[0,1]$	...	$\tilde{G}[0, 4]$
1	$\tilde{G}[1,0]$	$\tilde{G}[1,1]$	...	$\tilde{G}[1, 4]$
2	$\tilde{G}[2,0]$	$\tilde{G}[2,1]$	...	$\tilde{G}[2,4]$

3) 3-point DFTs of column

식 *의 내부 괄호 항 계산

$$X[k_2 5 + k_1] = \sum_{n_1=0}^4 \tilde{G}[n_2, k_1] W_{N_2}^{k_2 n_2}, \quad \begin{cases} 0 \leq k_1 \leq 4 \\ 0 \leq k_2 \leq 2 \end{cases}$$

$k_2 \backslash K_1$	0	1	...	4
0	X[0]	X[1]	...	X[4]
1	X[5]	X[6]	...	X[9]
2	X[10]	X[11]	...	X[14]





Windows & Window weighting

Why window weighting?

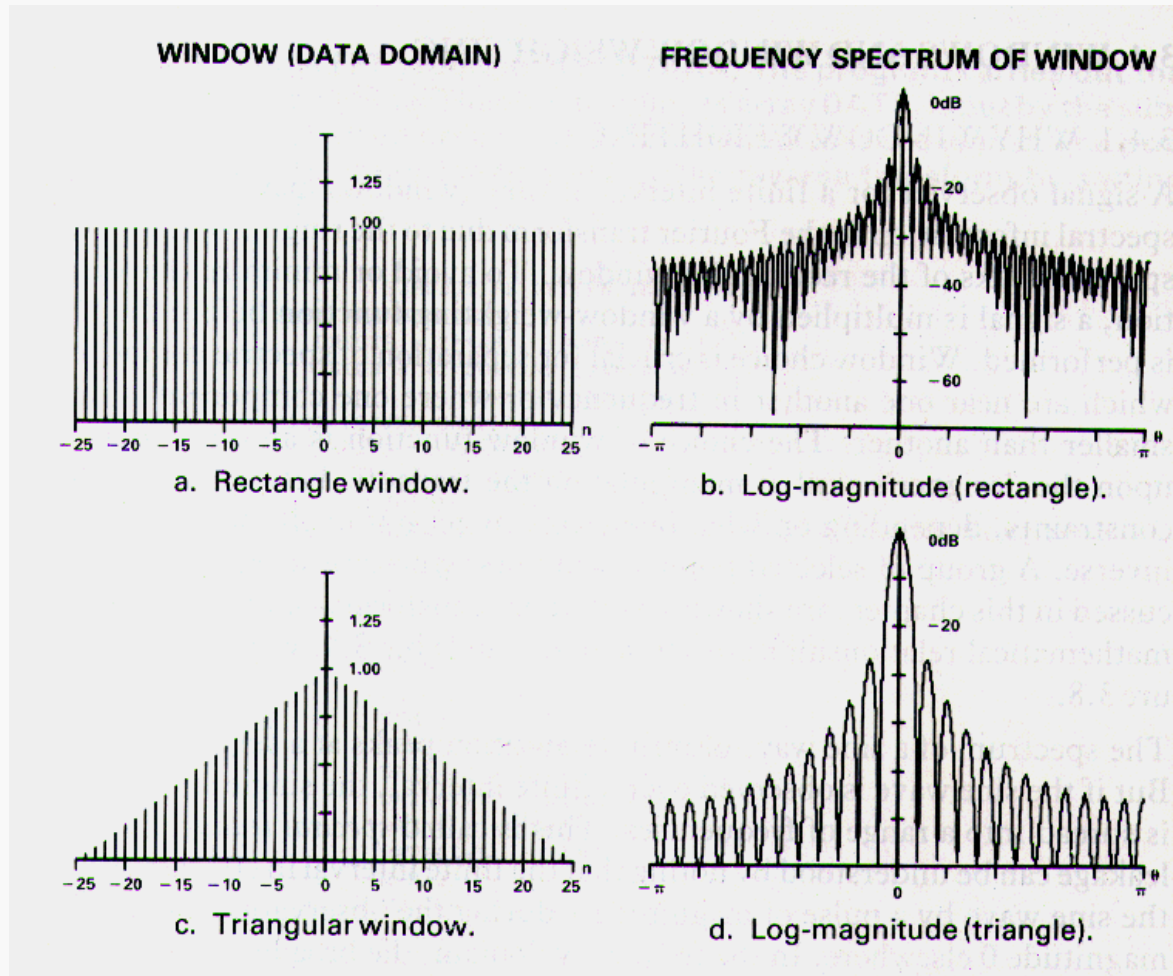
- ◇ A signal for finite interval of time

F.T → Distorted spectral information due to ringing of $\sin(f)/f$
spectral peaks of the rectangular window

- ◇ To minimize this distortion, a signal is multiplied by a window-weighting function before DFT is performed
- ◇ Window choice depends on designer's skill at manipulating the tradeoffs between the various constraints



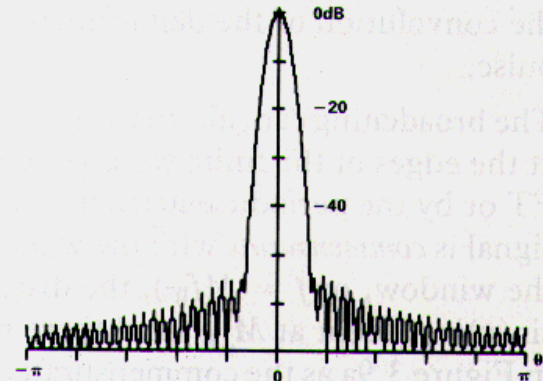
Window weighting function(1)



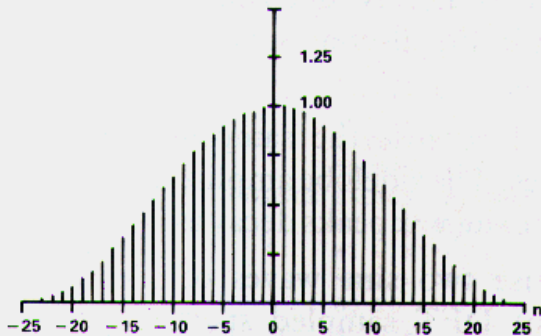
Window weighting function(2)



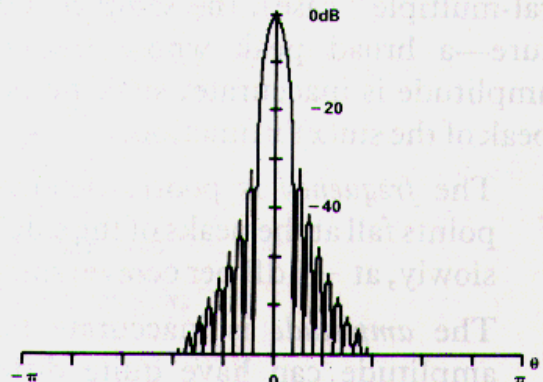
e. Gaussian window.



f. Log-magnitude (Gaussian, $\alpha = 3.0$).

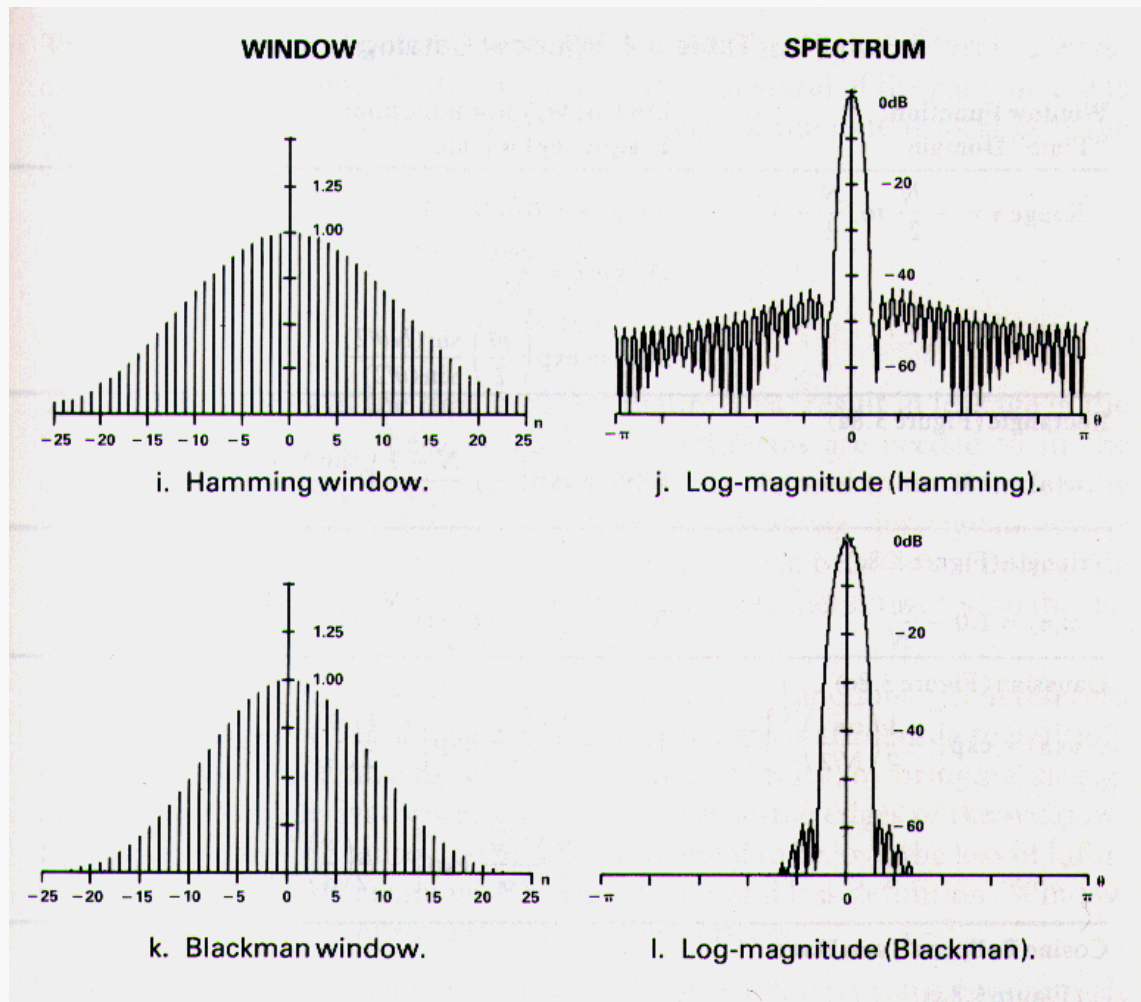


g. Cosine bell ($\cos^2$) window (also Hanning, Von Hann).

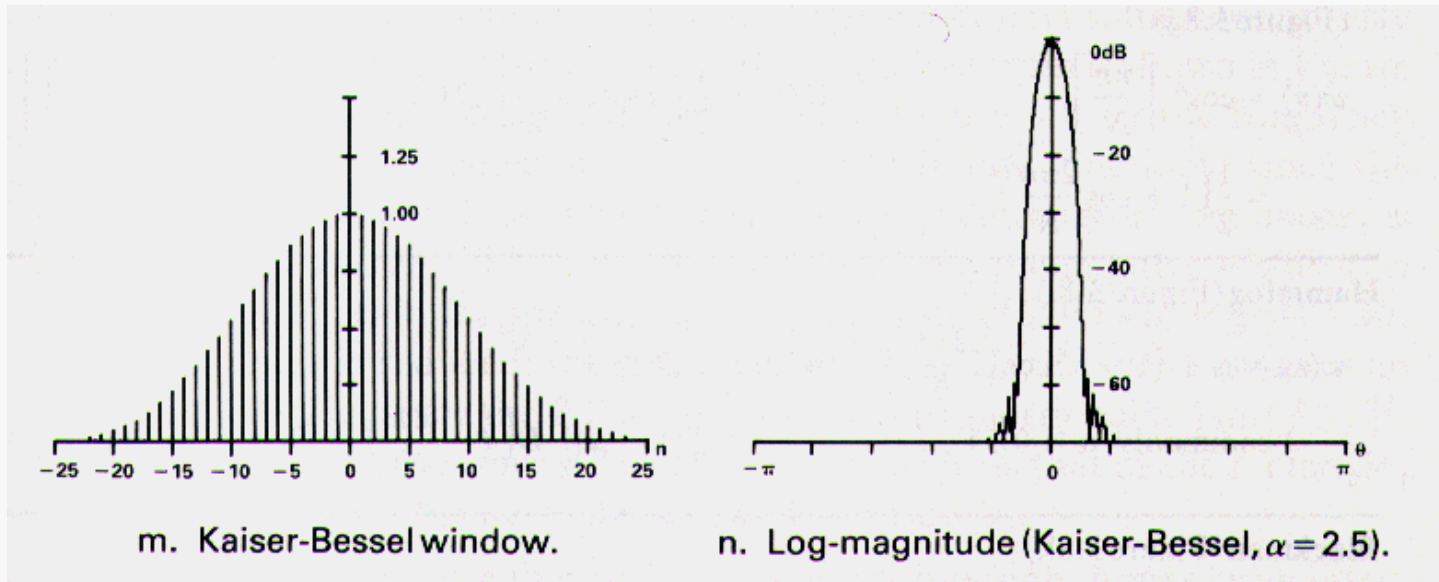


h. Log-magnitude (cosine bell).

Window weighting function(3)



Window weighting function(4)



Window catalog(1)

Table 3.4 Window Catalog

Window Function:
“Time” Domain

DFT of Window Function:
Frequency Domain

$$\text{Range } n = -\frac{N}{2} \text{ to } \frac{N}{2} - 1$$

$$\text{Range } k = 0 \text{ to } N - 1$$

$$\text{Define } \theta = \frac{2\pi k}{N}$$

$$D(\theta) = \exp\left(\frac{j\theta}{2}\right) \frac{\sin(N\theta/2)}{\sin(\theta/2)}$$

Rectangle (Figure 3.8a)

$$w(n) = 1.0$$

$$W(\theta) = \exp\left(-j \frac{N-1}{2} \theta\right) \frac{\sin(N\theta/2)}{\sin(\theta/2)}$$

Triangle (Figure 3.8c)

$$w(n) = 1.0 - \frac{|n|}{N}$$

$$W(\theta) = \frac{2}{N} \exp\left(-j \left(\frac{N}{2} - 1\right) \theta\right) \left[\frac{\sin N\theta/4}{\sin \theta/2} \right]^2$$

Gaussian (Figure 3.8e)

$$w(n) = \exp\left[-\frac{1}{2} \left(\frac{\alpha n}{N/2}\right)^2\right]$$

$$W(\theta) = \frac{1}{2} \frac{\sqrt{2\pi}}{\alpha} \exp\left[-\frac{1}{2} \left(\frac{\theta}{\alpha}\right)^2\right] \star D(\theta)$$

$$\approx \frac{N}{2} \frac{\sqrt{2\pi}}{\alpha} \exp\left[-\frac{1}{2} \left(\frac{\theta}{\alpha}\right)^2\right] \quad \begin{matrix} \alpha > 2.5 \\ \theta \text{ small} \end{matrix}$$



Window catalog(2)

Cosine Bell, $\cos^2$, von Hann

(Figure 3.8g)

$$w(n) = \cos^2 \left[\frac{\pi n}{N} \right]$$

$$= \frac{1}{2} \left(1 + \cos \frac{2\pi n}{N} \right)$$

$$W(\theta) = \frac{1}{2} \left\{ D(\theta) + \frac{1}{2} \left[D\left(\theta - \frac{2\pi}{N}\right) + D\left(\theta + \frac{2\pi}{N}\right) \right] \right\}$$

Hamming (Figure 3.8i)

$$w(n) = \alpha + (1 - \alpha) \cos \left(\frac{2\pi n}{N} \right)$$

commonly, $\alpha = 0.54$

$$W(\theta) = \alpha D(\theta) + \frac{1}{2} (1 - \alpha) \left[D\left(\theta - \frac{2\pi}{N}\right) + D\left(\theta + \frac{2\pi}{N}\right) \right]$$

Blackman (Figure 3.8k)

$$w(n) = 0.42 + 0.50 \cos \left(\frac{2\pi n}{N} \right)$$

$$+ 0.08 \cos \left(\frac{4\pi n}{N} \right)$$

$$W(\theta) = \sum (-1)^m \frac{\alpha_m}{2} \left[D\left(\theta - \frac{2\pi m}{N}\right) + D\left(\theta + \frac{2\pi m}{N}\right) \right]$$

commonly, $\alpha_0 = 0.42, \alpha_1 = 0.50, \alpha_2 = 0.08$

Kaiser (Figure 3.8m)

$$w(n) = \frac{I_0 \left[\pi \alpha \sqrt{1 - \left(\frac{n}{N/2} \right)^2} \right]}{I_0(\pi \alpha)}$$

$$W(\theta) \approx \frac{N}{I_0(\alpha \pi)} \frac{\sinh \sqrt{\alpha^2 \pi^2 - (N\theta/2)^2}}{\sqrt{\alpha^2 \pi^2 - (N\theta/2)^2}}$$



Window weighting summary(1)

- ◇ Triangle window
 - Computation simple: no sin/cos lookup table
 - Narrow spectral peak
 - Large sidelobes, only moderate falloff

- ◇ Cosine Bell window
 - Reasonable sidelobe rejection
 - Central peak as narrow as triangle
 - Computationally very efficient

- ◇ Hamming window
 - Moderately sharp central peak
 - Poor sidelobes



Window weighting summary(2)

◇ Blackman window

- Good sidelobe rejection
- Moderate computation: 2 cosines
- Broad central peak
- Excellent ENBW (Equivalent Noise BW) for spectral separation

◇ Kaiser window

- Best sidelobe rejection
- Computation unlike others above
- Design rules well established from filters
- Best ENBW for spectral separation





H/W design of FFT block

- Multiplier & Butterfly architecture
- FFT block design
 - Single BF Structure
 - Pipeline Structure
 - Parallel Processing Structure

H/W design of FFT block

◇ Multiplier design

- FFT 연산의 구성 요소 중 BF 구조를 위한 곱셈기 설계
- Serial multiplier, Booth multiplier 알고리즘 소개
- Radix-4 Booth 알고리즘을 이용한 16×16 곱셈기 설계

◇ Three kinds of FFT block design

- Single BF structure
- Pipeline structure
- Parallel processing structure



Design consideration of FFT block

- ◇ Number representation
 - Fixed-point or floating point, sign magnitude or 2's complement
- ◇ Coefficient 사용 방식
 - Look-table(ROM)을 이용한 방식은 빠르지만 extra memory가 요구됨
 - 각 stage에 필요한 계수의 규칙성을 이용한 Recursion 공식을 통한 계산은 memory가 적게 요구되나 Low speed임
- ◇ Coefficient Quantization
 - 계수 양자화 잡음을 고려, finite # of bit을 결정
- ◇ Scaling of internal nodes(over/underflow)
 - Internal finite # of bit에 의한 over/underflow를 제거하는 방안
 - 일반적으로 각 stage마다 1/2씩 scale down
- ◇ Overall speed, size를 고려하여 적절한 parallelism 이용
 - Parallel 연산을 위한 적절한 알고리즘 선택
 - Radix-2 or Radix-4 방식, *in-place* or *not-in-place* 방식



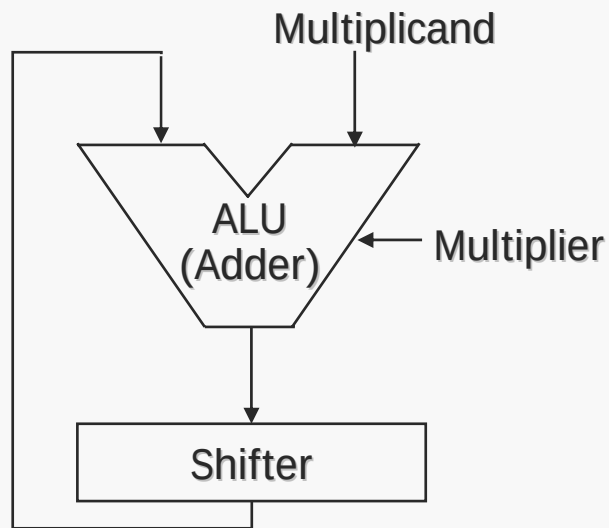
Serial multiplier

• 계산 방법

$$\begin{array}{r}
 01011 \\
 \times 10011 \\
 \hline
 01011 \\
 01011 \\
 00000 \\
 00000 \\
 + 01011 \\
 \hline
 011010001
 \end{array}$$

Multiplicand

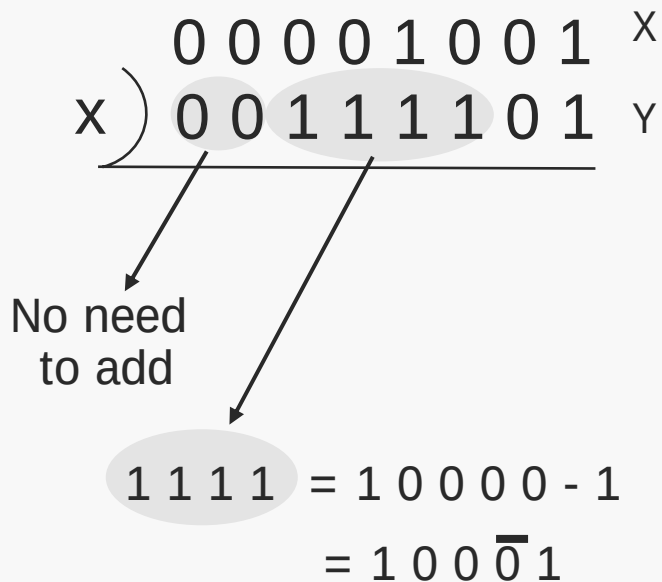
Multiplier



- 문제점 : Low speed



Radix-4 Booth's Algorithm



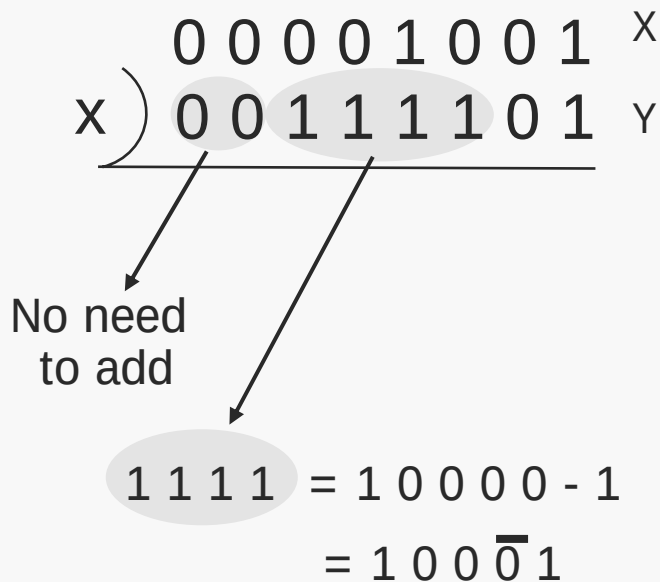
Radix-4 Algorithm

Y_{i+1}	Y_i	Y_{i-1}	복호	Partial product
0	0	0	0 0	$+0X$ (No string)
0	0	1	0 1	$+1X$ (End of string)
0	1	0	0 1	$+1X$ (A string)
0	1	1	1 0	$+2X$ (End of string)
1	0	0	- 1 0	$- 2X$ (Beginning of string)
1	0	1	- 1 1	$- 1X$ ($-2X + X$)
1	1	0	$\overline{0}$ 1	$- 1X$ (Beginning of string)
1	1	1	0 0	$- 0X$ (Center of string)

- String of 0's
 - String of 1's
- can be simplified



Radix-4 Booth's Algorithm



Radix-4 Algorithm

Y_{i+1}	Y_i	Y_{i-1}	복호	Partial product
0	0	0	0 0	$+0X$ (No string)
0	0	1	0 1	$+1X$ (End of string)
0	1	0	0 1	$+1X$ (A string)
0	1	1	1 0	$+2X$ (End of string)
1	0	0	- 1 0	$- 2X$ (Beginning of string)
1	0	1	- 1 1	$- 1X$ ($-2X + X$)
1	1	0	$\overline{0}$ 1	$- 1X$ (Beginning of string)
1	1	1	0 0	$- 0X$ (Center of string)

- String of 0's
 - String of 1's
- can be simplified



Example

(X = 93)

(Y = 106)

$$(0\ 1\ 0\ 1\ 1\ 1\ 0\ 1)_2 \times (0\ 1\ 1\ 0\ 1\ 0\ 1\ 0)_2 = 9858$$

Sign extension

X

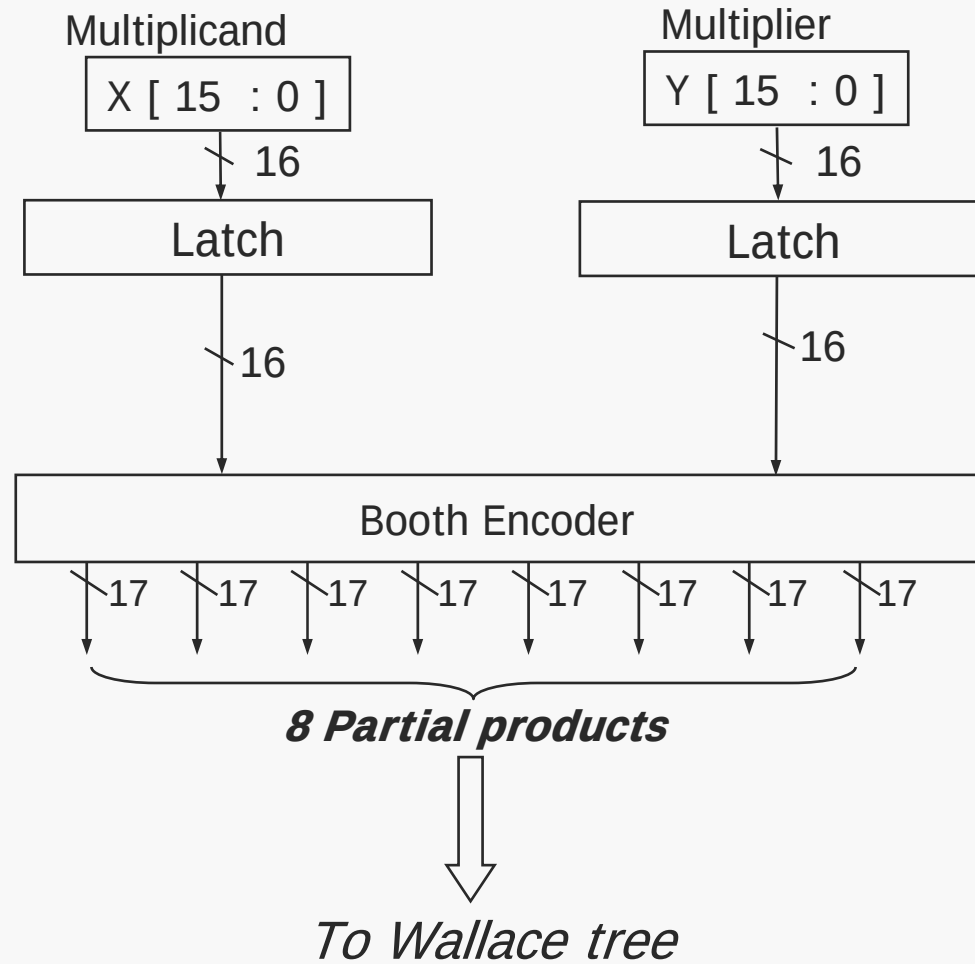
$$\begin{array}{r}
 \begin{array}{cccccccc}
 0 & 1 & 0 & 1 & 1 & 1 & 0 & 1 \\
 \begin{array}{cc} +2X & -1X \end{array} \\
 \begin{array}{cccccccc}
 0 & 1 & 1 & 0 & 1 & 0 & 1 & 0 & 0 \\
 \begin{array}{cc} -1X & -2X \end{array}
 \end{array}
 \end{array}
 = Y$$

$$\begin{array}{r}
 \begin{array}{cccccccccccccccc}
 1 & 1 & 1 & 1 & 1 & 1 & 1 & 0 & 1 & 0 & 0 & 0 & 1 & 1 & 0 & \dots & -2X \\
 1 & 1 & 1 & 1 & 1 & 1 & 1 & 0 & 1 & 0 & 0 & 0 & 1 & 1 & & \dots & -1X \\
 1 & 1 & 1 & 1 & 0 & 1 & 0 & 0 & 0 & 1 & 1 & & \dots & & & \dots & -1X \\
 + & 0 & 1 & 0 & 1 & 1 & 1 & 0 & 1 & 0 & & \dots & & & & \dots & +2X
 \end{array}
 \end{array}$$

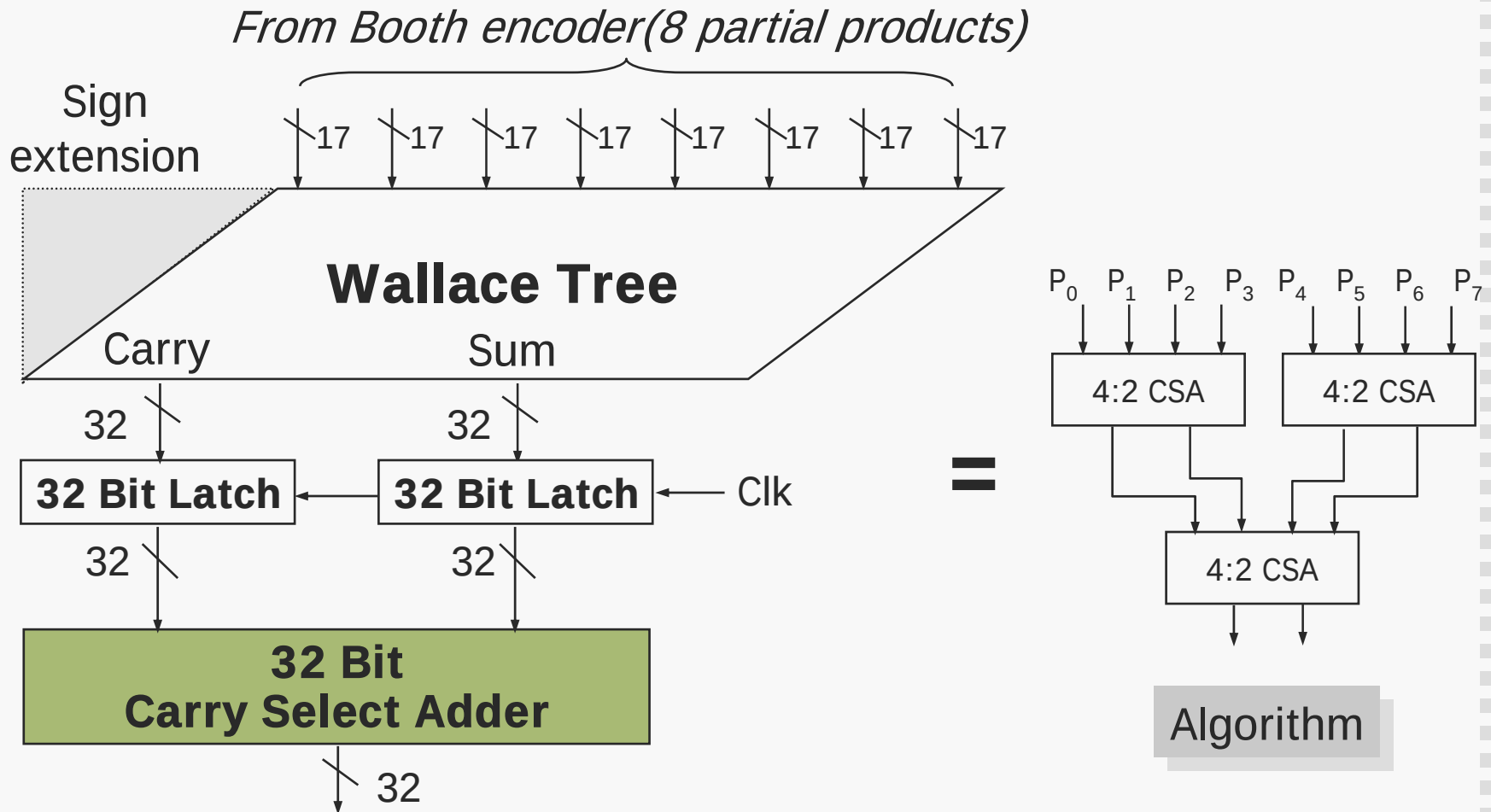
0 1 0 0 1 1 0 1 0 0 0 0 0 1 0 **(9858)**



16X16 Multiplier Block Diagram(1)

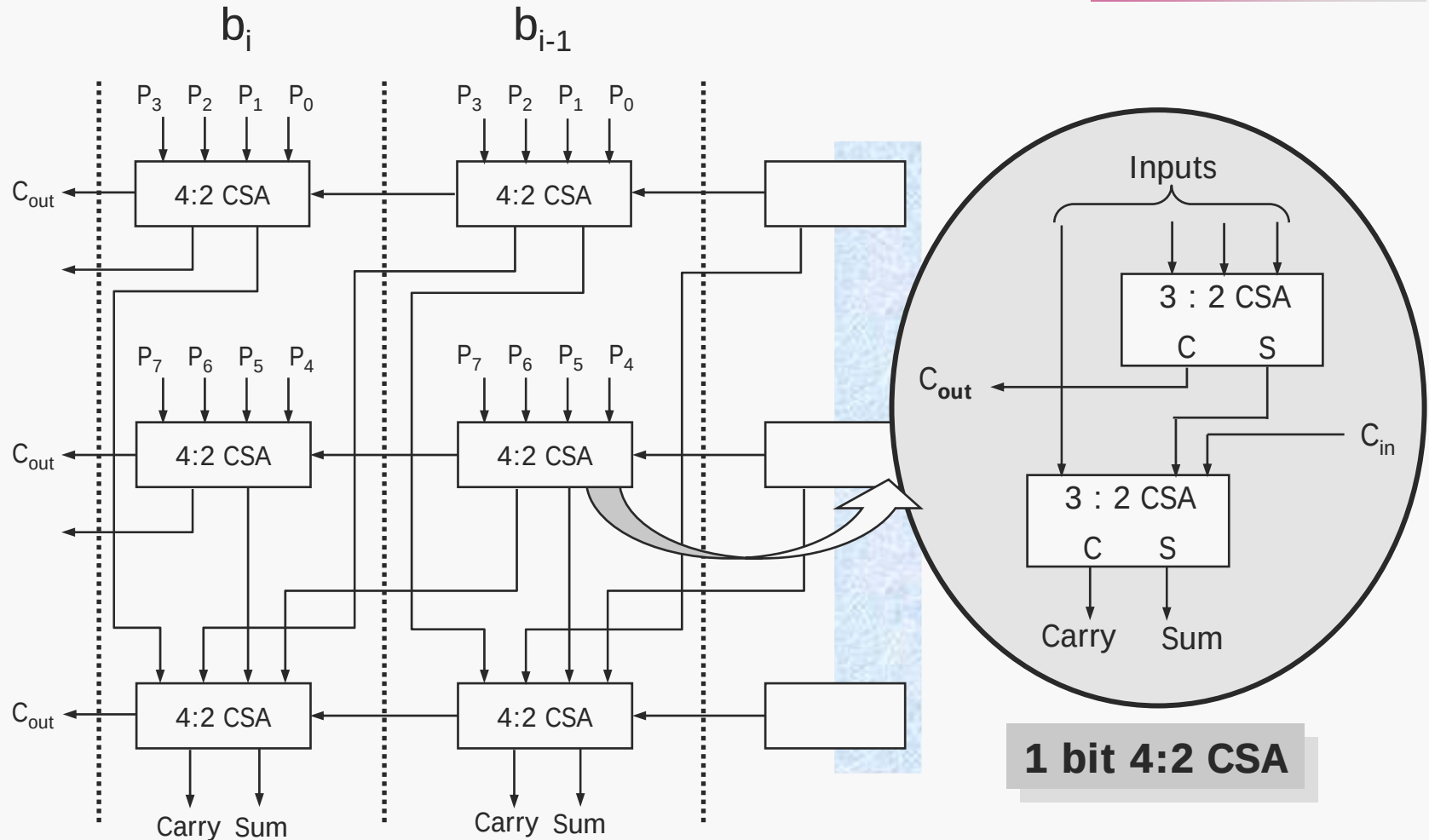


16X16 Multiplier Block Diagram(2)



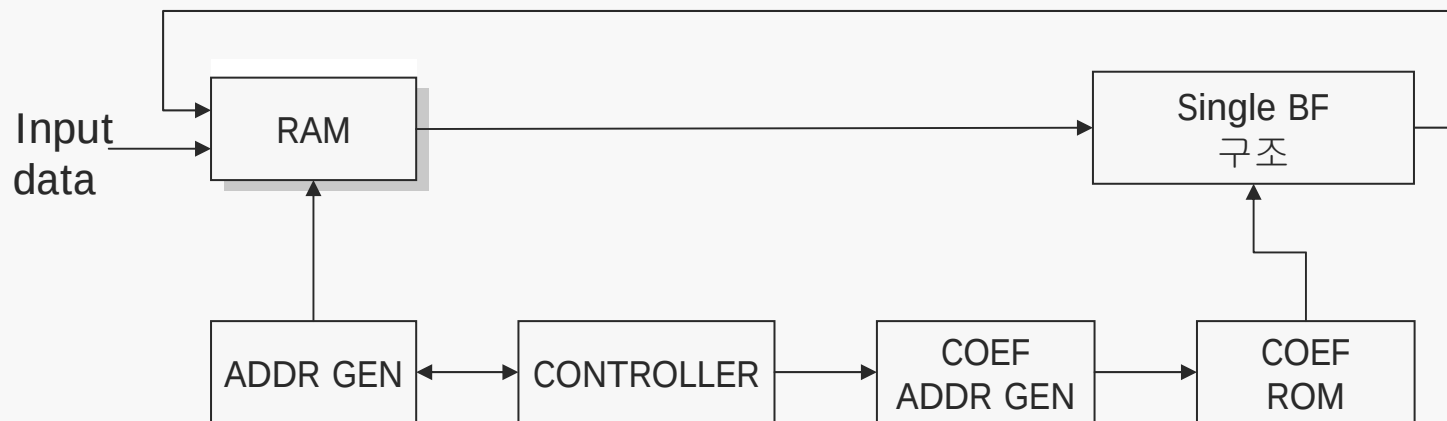
비규칙성을 없애기 위해

4 : 2 CSA를 사용한 Wallace Tree



Single BF Structure(1)

- ◇ Single BF를 사용한 FFT 구현
 - FFT Flow graph상에서 사용되는 $(N/r\log rN)$ 개의 BF 연산을 하나로 구현
 - In-place 알고리즘을 이용하기 위해 시간 영역 또는 주파수 영역에서 Bit-reversing이 필요
 - 계수는 ROM에 저장하여 사용
- ◇ Single BF를 사용한 FFT Processor Block Diagram



Single BF Structure(2)

◇ Radix-4 DIF single BF FFT processor

■ Radix-4 DIF BF equation

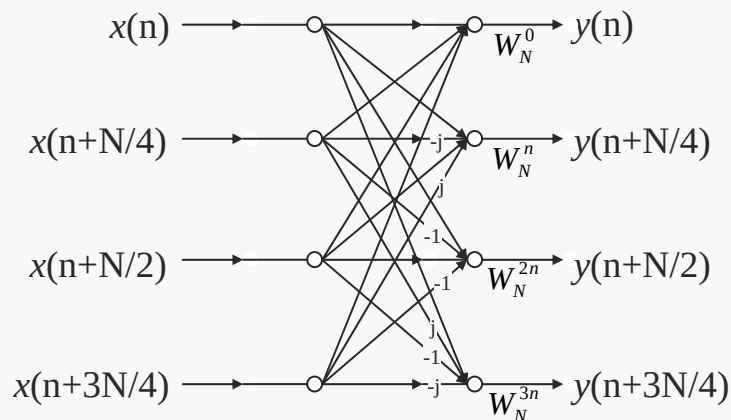
$$X[4k] = \sum_{n=0}^{(N/4)-1} [x(n) + x(n + N/4) + x(n + N/2) + x(n + 3N/4)] W_N^0 W_{N/4}^{nk}$$

$$X[4k+1] = \sum_{n=0}^{(N/4)-1} [x(n) - jx(n + N/4) - x(n + N/2) + jx(n + 3N/4)] W_N^n W_{N/4}^{nk}$$

$$X[4k+2] = \sum_{n=0}^{(N/4)-1} [x(n) - x(n + N/4) + x(n + N/2) - x(n + 3N/4)] W_N^{2n} W_{N/4}^{nk}$$

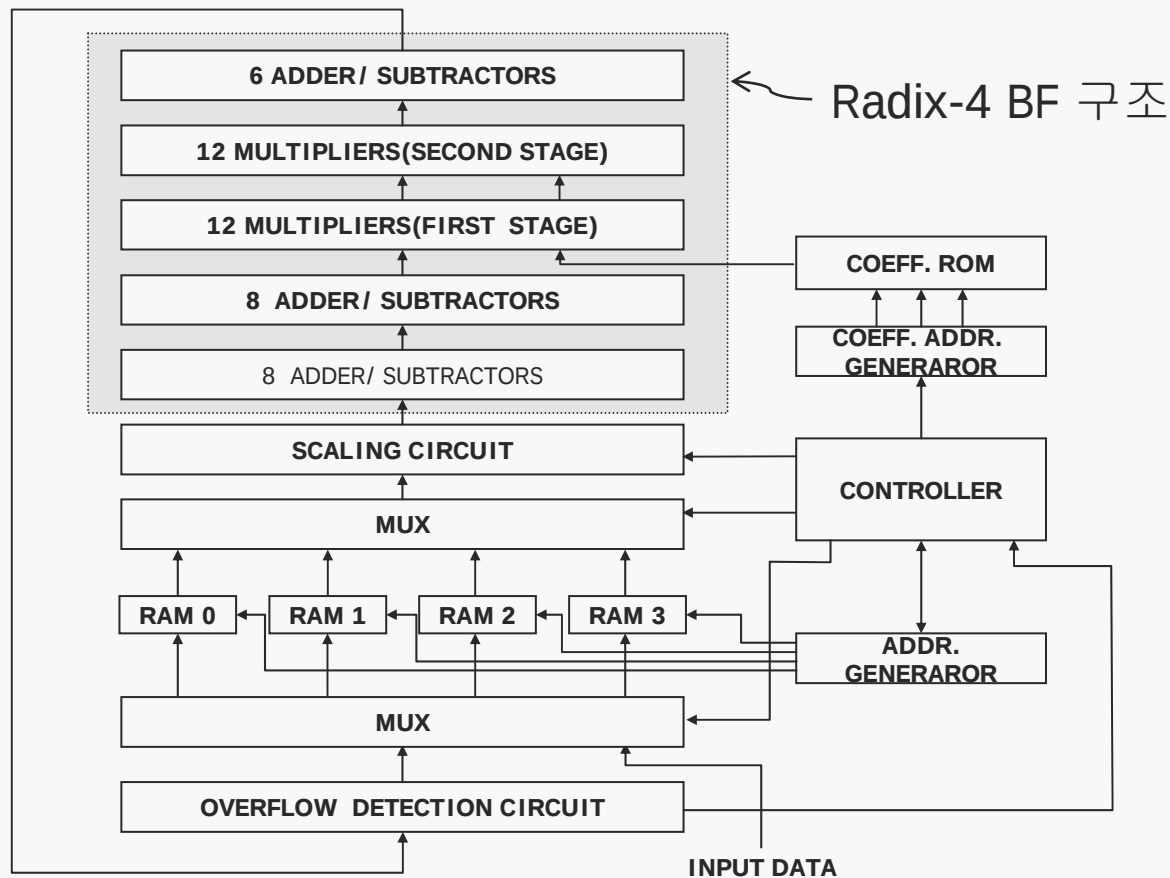
$$X[4k+3] = \sum_{n=0}^{(N/4)-1} [x(n) + jx(n + N/4) - x(n + N/2) - jx(n + 3N/4)] W_N^{3n} W_{N/4}^{nk}$$

■ Radix-4 DIF BF Flow graph



Single BF Structure(3)

- Radix-4 DIF Single BF FFT processor 구조의 상세 block diagram



Single BF Structure(4)

◇ Address Generator

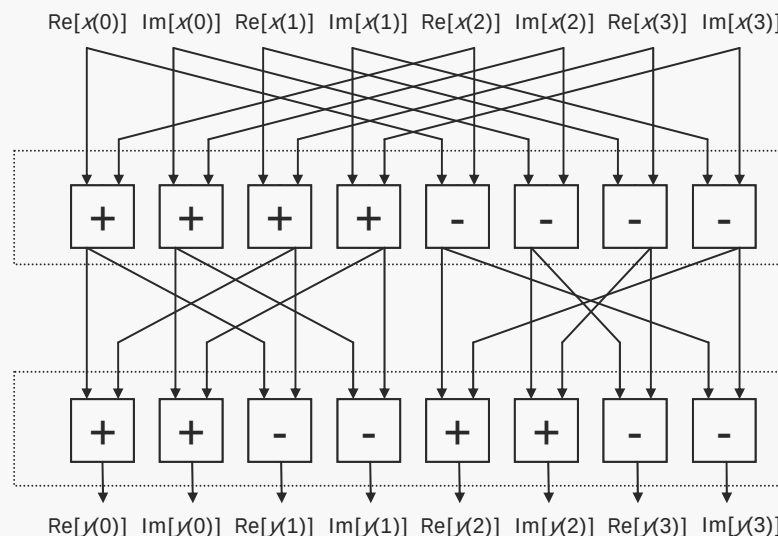
- BF counter, Stage counter를 이용하여 BF 입력/출력 Address를 결정
- In-place 알고리즘을 사용하기 때문에 Read/Write의 Address는 동일

◇ Coefficient Address Generator

- Radix-4 BF 연산에 필요한 3개 계수의 Address를 결정

◇ Radix-4 BF 구조의 ADD/SUB 구현

- 12개 complex ADD/SUB의 real ADD/SUB 구현



Single BF Structure(5)

- ◇ Radix-4 BF 구조의 complex multiplier 구현
 - One complex multiplication = four real multiplication + two add/sub
 - 3개 complex multiplication을 12개 real multiplication 과 6개 real add/sub로 구현

- ◇ Sign-Bit Detection Circuit and Scaling Circuit
 - BF 연산 결과 overflow가 생길 때 truncation/shift right 수행 후 중간 결과 저장
 - Overflow detection circuit을 통해 각 stage에서 shift right number detection 수행
 - Scaling circuit에서 previous stage의 # of shift right 만큼 shift left를 수행한 후, BF에 입력하여 BF 연산의 dynamic range를 넓힘

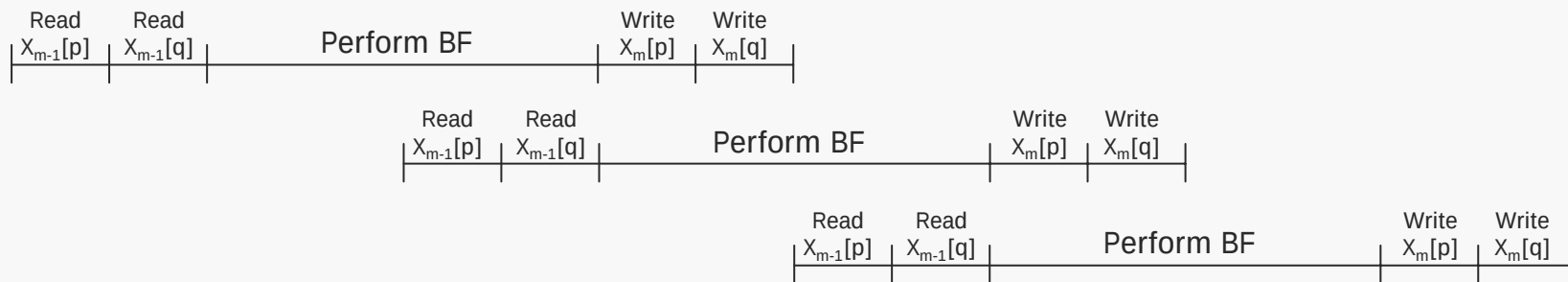


Single BF Structure(6)

◆ FFT의 연산 속도를 높이는 방법

1) Time overlapping of BF operation and memory access(Single BF 사용)

ex) Timing diagram for “optimum”Radix-2



2) Higher radix 구조의 사용

3) $\log_2 N$ 개의 BF를 사용한 Pipeline FFT 구조 사용

4) $N/2$ 개 BF를 사용한 Parallel Processors FFT 구조 사용

5) Flow graph에 나타난 $N/2 \log_2 N$ 모든 BF 구조 개수 만큼 BF를 사용한 super-parallel processors FFT 구조 사용



Pipeline Structure(1)

◇ Pipeline FFT processor

- 각 Stage마다 BF를 두어 DFT를 parallel하게 계산
- Non-stopping processing → lower clock frequency 사용 가능
→ high speed processing or low power 가능
- 구조의 규칙성으로 인해 VHDL을 이용한 설계 용이

◇ 종류

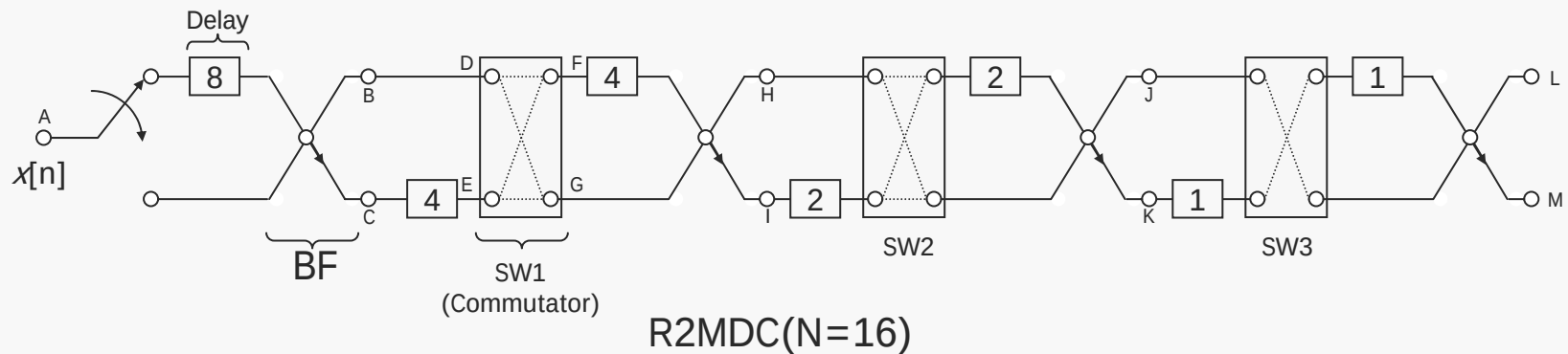
- R2MDC(Radix-2 Multi-path Delay Commutator)
 - Input sequence를 two parallel data stream으로 나누고 delay element (register)를 이용하여 두 data element의 distance를 조정하여 BF에 입력
- R2SDF(Radix-2 Single-path Delay Feedback)
 - BF output의 일부를 feedback shift register에 저장함으로 memory size를 줄임
- R4MDC(Radix-4 Multi-path Delay Commutator)
 - R2MDC의 Radix-4 version
- R4SDF(Radix-4 Single-path Delay Feedback)
 - R2SDF의 Radix-4 version
- R4SDC(Radix-4 Single-path Delay Commutator)
 - Single path, BF 당 Multiplier 하나



Pipeline Structure(2)

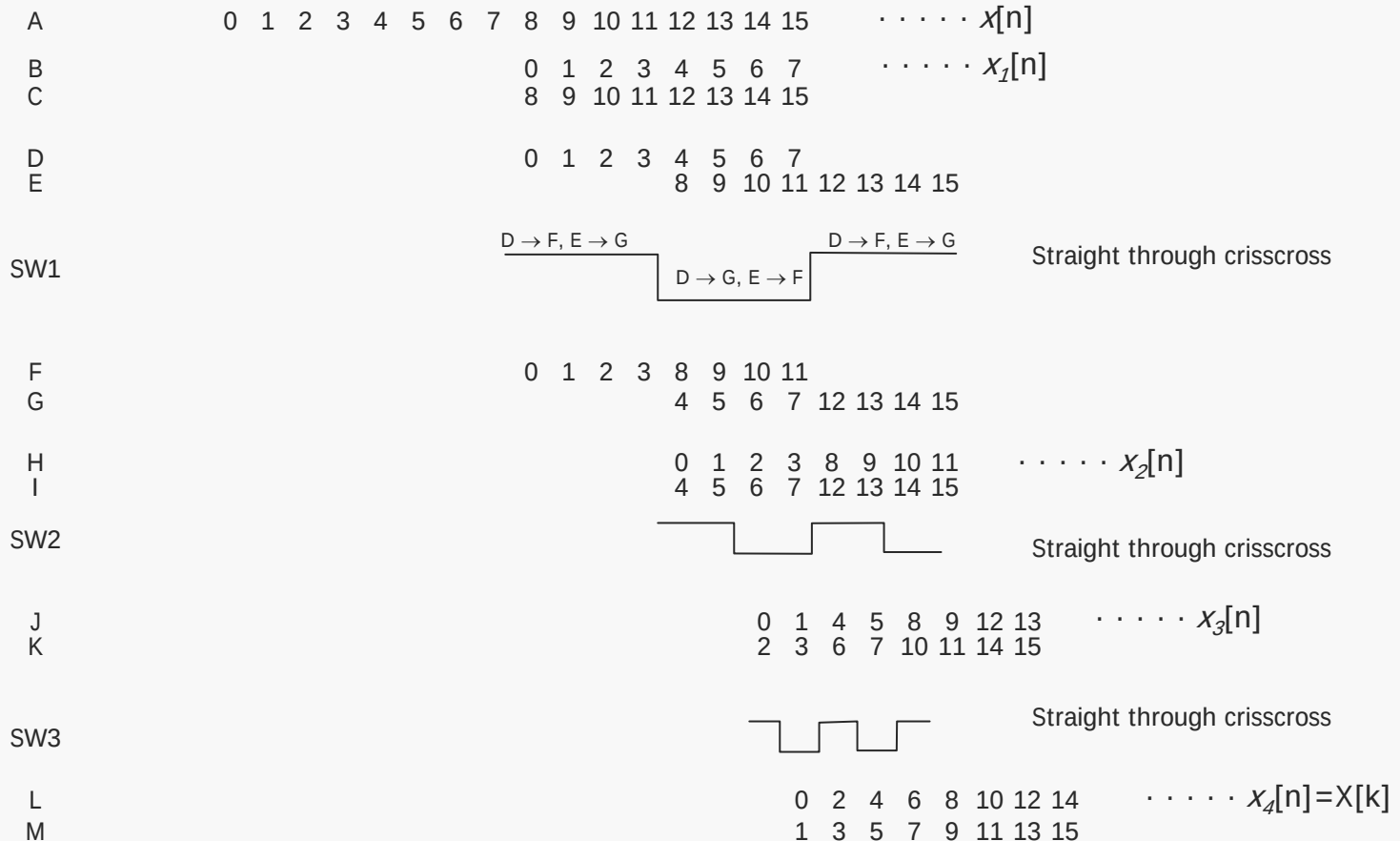
◇ R2MDC 방식

- Delay element (shift register)는 previous stage의 1/2
- Arithmetic elements (BF)는 1/2 time period 동안 busy
- 각 stage의 switch는 앞 stage switch보다 두 배 빠르게 switching 수행
- Output is bit-reversed



Pipeline Structure(3)

◇ R2MDC 방식을 사용한 16-point DIF 계산 과정



Pipeline Structure(4)

◇ R2SDF 방식

- BF module, multiplier, shift register로 구성
- N=16인 경우 first stage에서 8 inputs을 register에 저장
- 다음 input부터 아래 값을 연산

$$b_i = a_i + a_{i+8}$$

$$b_{i+8} = a_i - a_{i+8} \quad \text{where } i=0, 1, \dots, 6, 7$$

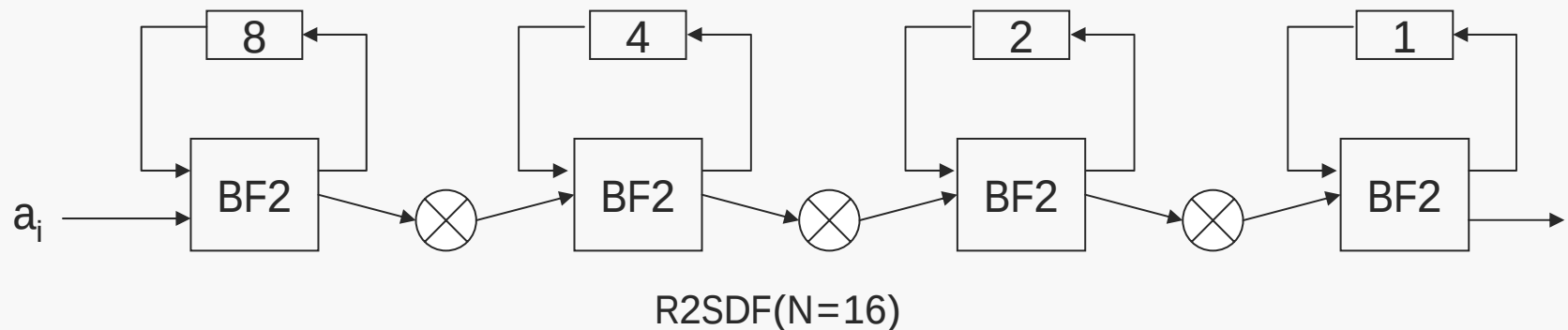
- b_i 는 Coeff.와 곱셈 수행, b_{i+8} shift register에 feedback
- b_i 는 Coeff.와 곱셈 수행 종료 후, b_{i+8} 를 꺼내 Coeff.와 곱셈 수행(stage 종료)
- 이때 다음 stage에서는 BF연산에 필요한 Data들이 모두 loading 되어 있어 대기 시간 없이 BF 연산 수행 가능



Pipeline Structure(5)

◇ R2SDF 방식(계속)

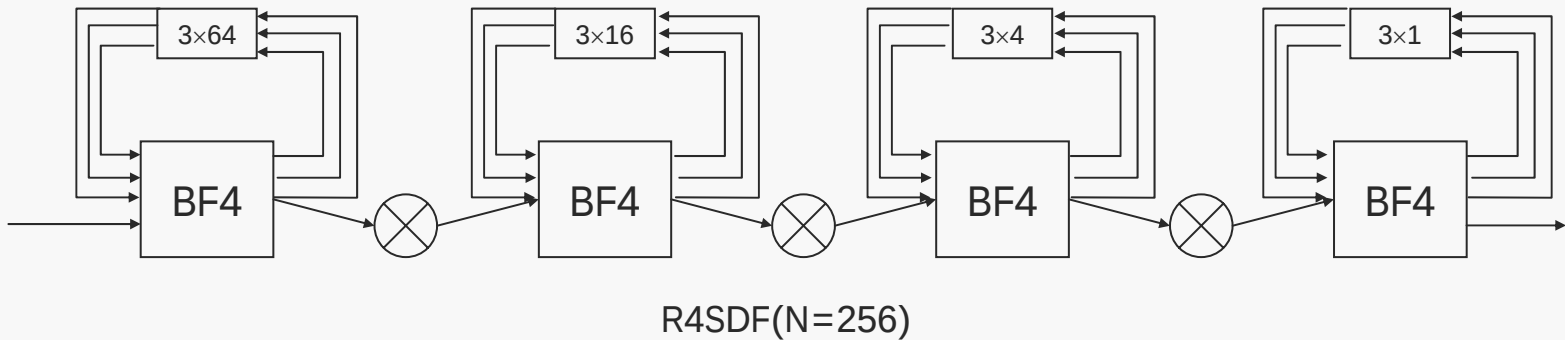
- 다음 stage에서도 동일한 방법으로 BF 연산 수행
- R2MDC와 비교하여 Multiplier와 Adder 개수는 같지만 memory size는 $3N/2 - 2$ 에서 $N-1$ 개로 준다



Pipeline Structure(6)

◇ R4SDF 방식

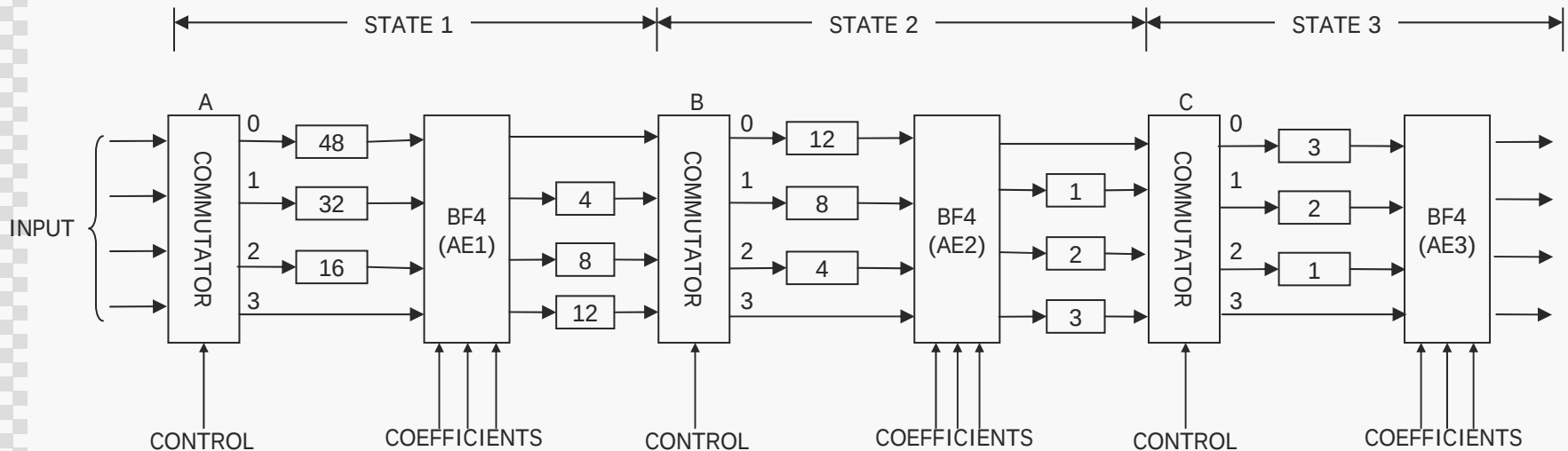
- R2SDF의 radix-4 version
- BF4 output의 4개 중 3개는 Feedback



Pipeline Structure(7)

◇ R4MDC 방식

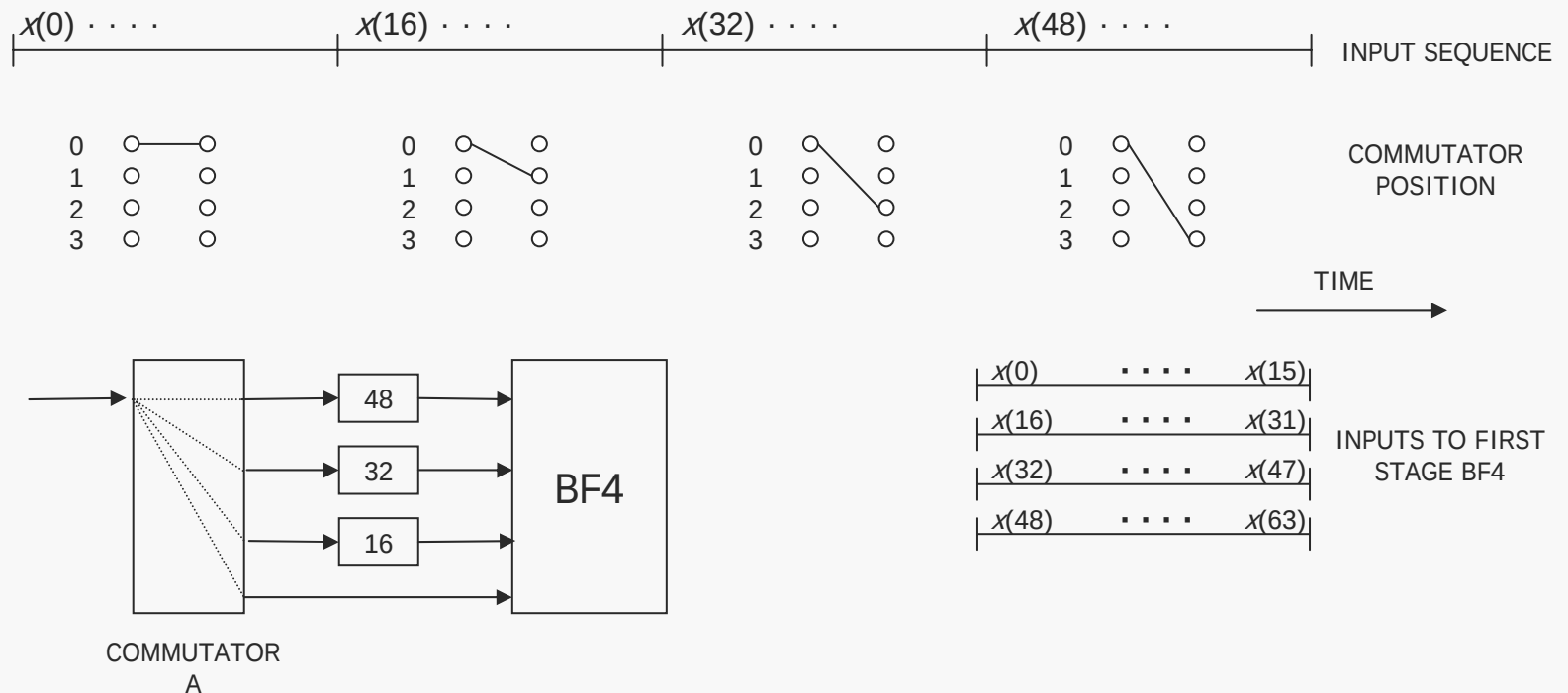
- BF4 : 3 complex multiplier + 8 add/sub
- Commutator : four-input, four-output switch
- Delay element : shift register
- N=64인 경우 전체 Block diagram



Pipeline Structure(8)

◇ R4MDC 방식(계속)

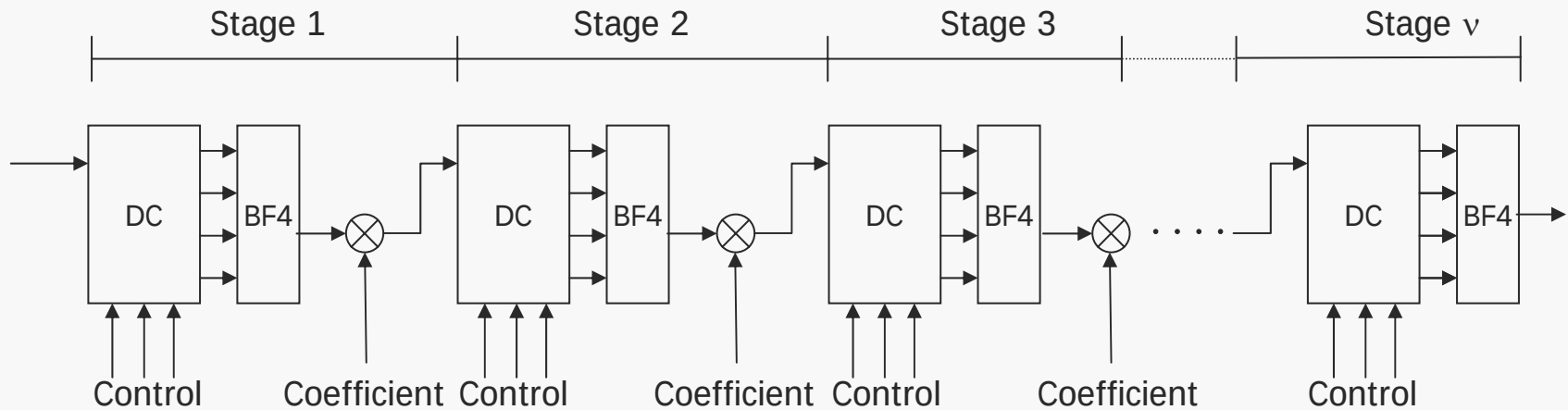
- Commutator A는 매 16 sample 마다 switching
- 64개 전체 inputs을 4개로 나누어 BF4에 입력한다



Pipeline Structure(9)

◇ R4SDC 방식

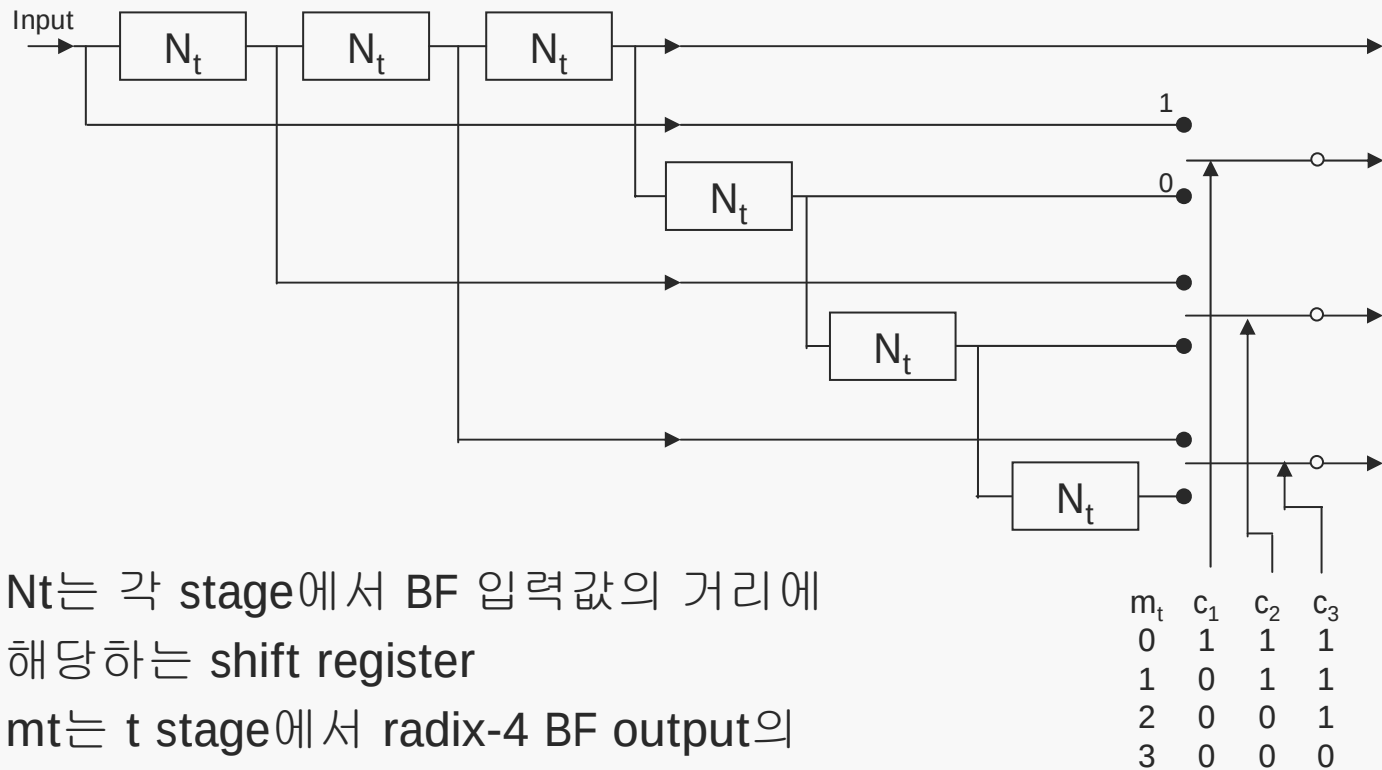
- Split radix-4 BF into simplified radix-4 BF
- Compute only one of original 4 outputs each time



R4SDC scheme

Pipeline Structure(10)

◇ Commutator for stage t

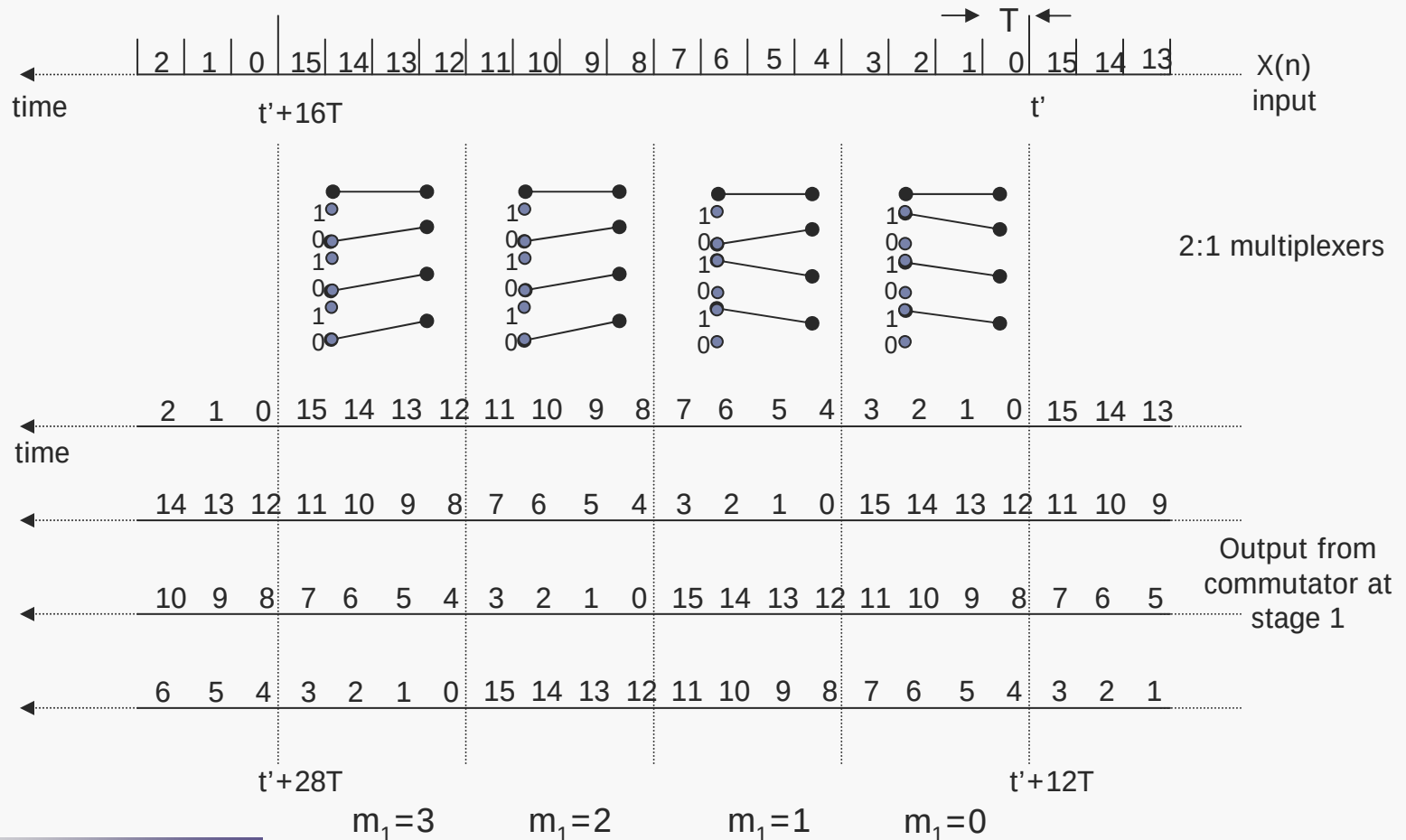


- N_t 는 각 stage에서 BF 입력값의 거리에 해당하는 shift register
- m_t 는 t stage에서 radix-4 BF output의 top value에서 bottom Value를 나타냄



Pipeline Structure(11)

◇ Timing and operation of commutator at stage 1 for 16-point



Pipeline Structure(12)

◇ Pipeline FFT 구조의 비교

	Multiplier #	Adder #	Memory size	Control
R2MDC	$2(\log_4 N - 1)$	$4\log_4 N$	$3N/2 - 2$	Simple
R2SDF	$2(\log_4 N - 1)$	$4\log_4 N$	$N - 1$	Simple
R4SDF	$\log_4 N - 1$	$8\log_4 N$	$N - 1$	Medium
R4MDC	$3(\log_4 N - 1)$	$8\log_4 N$	$5N/2 - 4$	Simple
R4SDC	$\log_4 N - 1$	$3\log_4 N$	$2N - 2$	Complex

- R4SDF 방식은 R4SDC 방식에 비해 memory size가 작지만 Adder로 구성된 BF가 복잡
- Delay-feedback approach가 Delay-commutator approach보다 메모리 이용면에서 효율적
- Radix-2 approach가 Radix-4 approach에 비해 BF 구조가 간단



Parallel processing Structure

◇ Parallel processing 방식

- 하나의 stage를 계산하는데 필요한 BF를 모두 만들어 각 stage를 Parallel 하게 수행
- 구현 방법에는 Systolic Array, Perfect shuffle, FFT Network 등이 있음
- 여기서는 2 Dimensional Systolic Array를 이용한 방식과 Perfect shuffle을 이용한 방식을 살펴봄

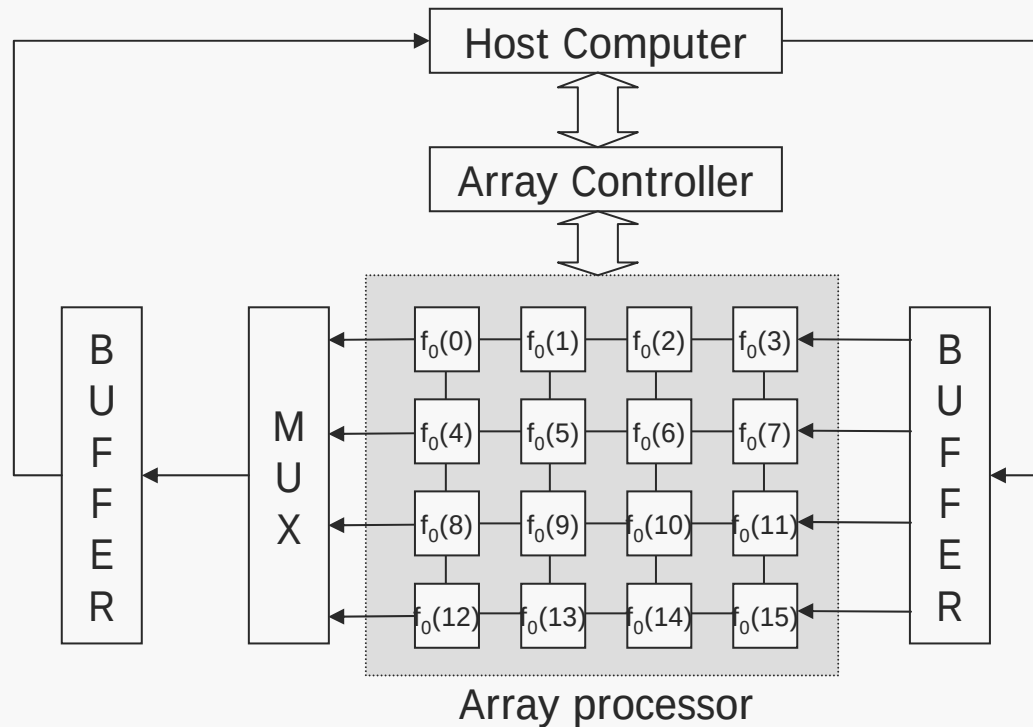
◇ Systolic Array processors의 특징

- Simple Cell로 구성
- 단순하고 규칙적인 데이터 흐름
- Local communication
- 규칙적인 확장성(modular expansion)
- 고도의 병렬성(concurrency)과 pipeline 처리
- 반복적인 연산으로 구성된 알고리즘의 효율적



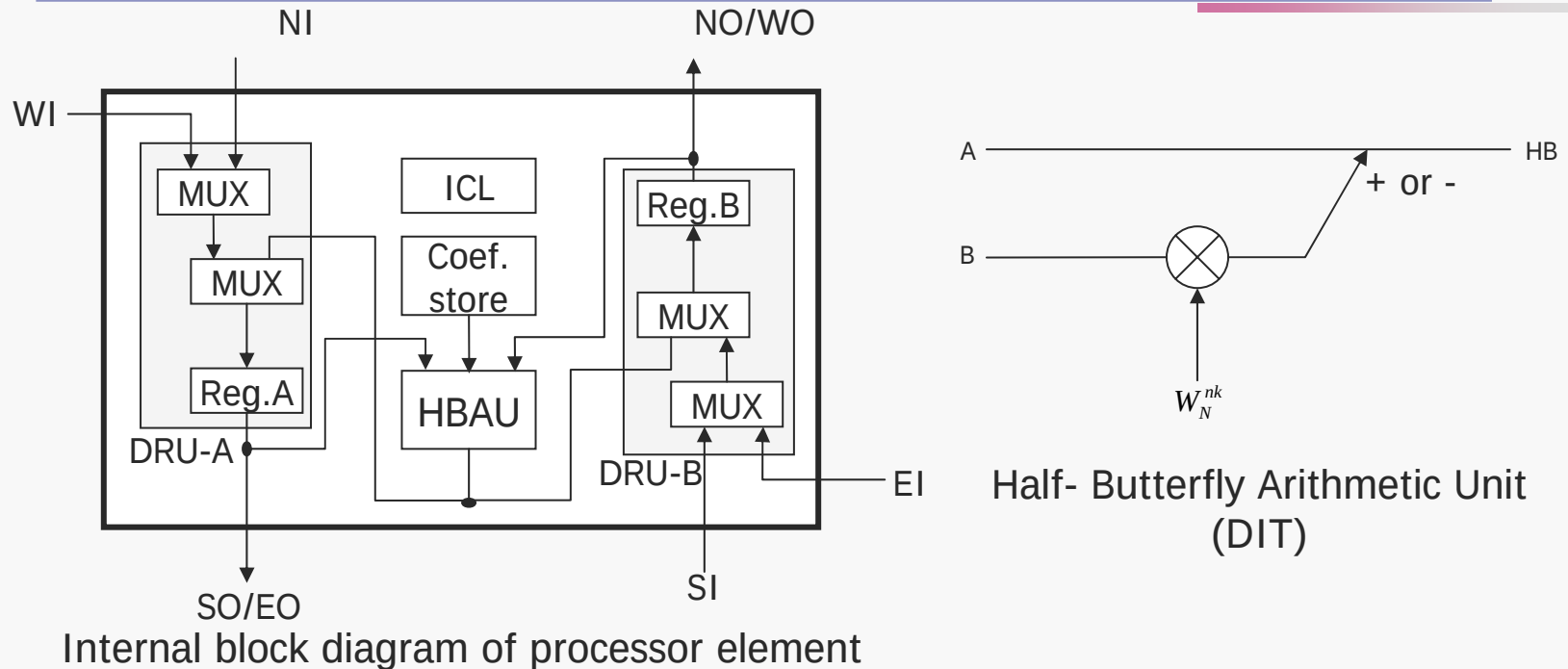
Systolic Array Processor(1)

◆ 전체 시스템 구성



- FFT연산을 Stage마다 Parallel 하게 연산하는 Array processor, 입력/출력 버퍼, Controller로 구성
- 16-point FFT 연산인 경우 4×4 processor로 구성

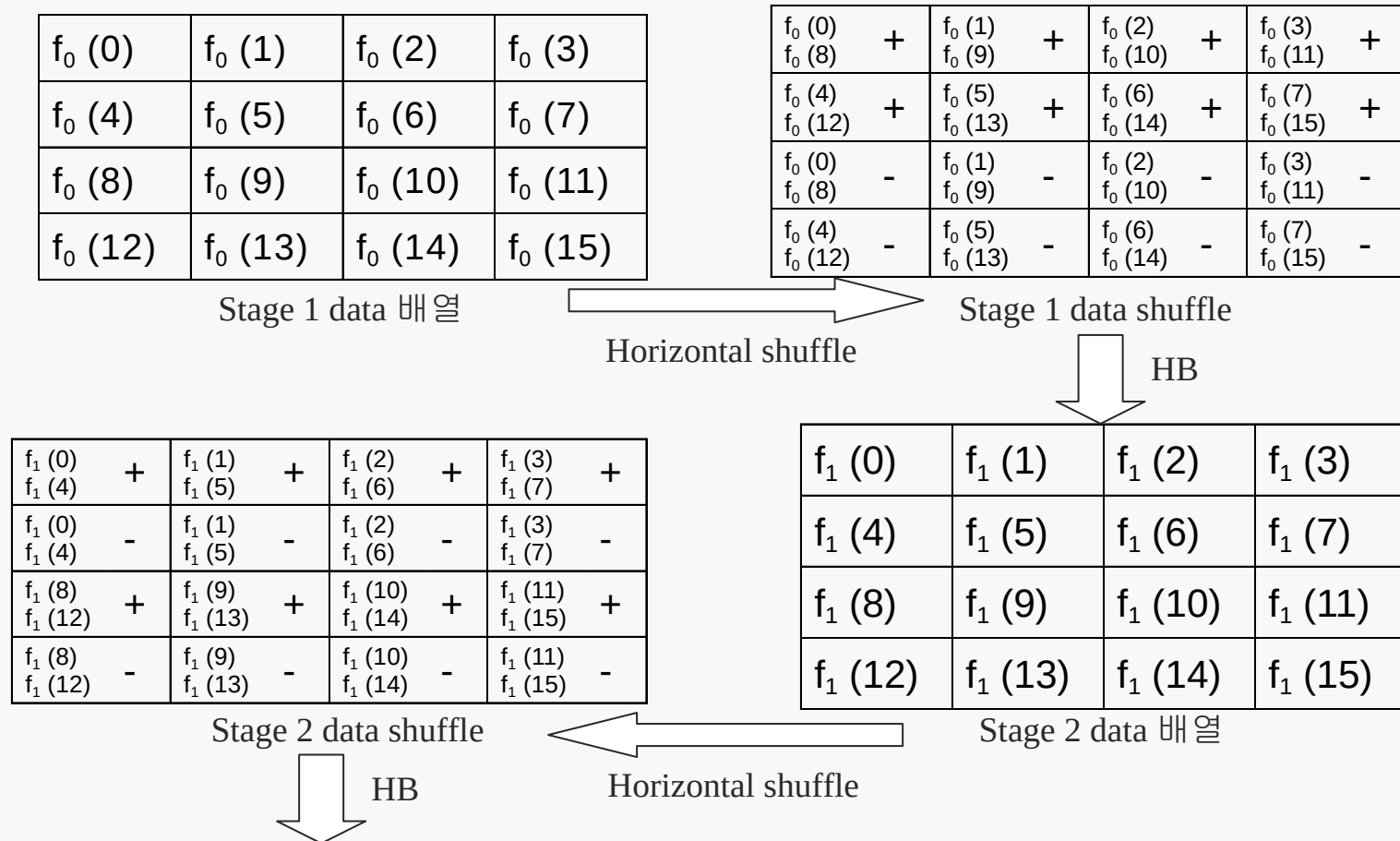
Systolic Array Processor (2)



- ◇ DRU(Data Routing Unit)은 인접한 4개의 PE와 data shuffling 하기 위해 Data communication을 수행
- ◇ MUX로 Vertical/Horizontal data shuffling 제어

Systolic Array Processor (3)

◇ Stage 별 PE의 data 배열 및 shuffle 과정



Systolic Array Processor (4)

$f_2(0)$	$f_2(1)$	$f_2(2)$	$f_2(3)$
$f_2(4)$	$f_2(5)$	$f_2(6)$	$f_2(7)$
$f_2(8)$	$f_2(9)$	$f_2(10)$	$f_2(11)$
$f_2(12)$	$f_2(13)$	$f_2(14)$	$f_2(15)$

Stage 3 data 배열

Vertical shuffle

$f_2(0)$	+	$f_2(1)$	+	$f_2(0)$	-	$f_2(1)$	-
$f_2(2)$		$f_2(3)$		$f_2(2)$		$f_2(3)$	
$f_2(4)$	+	$f_2(5)$	+	$f_2(4)$	-	$f_2(5)$	-
$f_2(6)$		$f_2(7)$		$f_2(6)$		$f_2(7)$	
$f_2(8)$	+	$f_2(9)$	+	$f_2(8)$	-	$f_2(9)$	-
$f_2(10)$		$f_2(11)$		$f_2(10)$		$f_2(11)$	
$f_2(12)$	+	$f_2(13)$	+	$f_2(12)$	-	$f_2(13)$	-
$f_2(14)$		$f_2(15)$		$f_2(14)$		$f_2(15)$	

Stage 3 data shuffle

HB

$f_3(0)$	+	$f_3(0)$	-	$f_3(2)$	+	$f_3(2)$	-
$f_3(1)$		$f_3(1)$		$f_3(3)$		$f_3(3)$	
$f_3(4)$	+	$f_3(4)$	-	$f_3(6)$	+	$f_3(6)$	-
$f_3(5)$		$f_3(5)$		$f_3(7)$		$f_3(7)$	
$f_3(8)$	+	$f_3(8)$	-	$f_3(10)$	+	$f_3(10)$	-
$f_3(9)$		$f_3(9)$		$f_3(11)$		$f_3(11)$	
$f_3(12)$	+	$f_3(12)$	-	$f_3(14)$	+	$f_3(14)$	-
$f_3(13)$		$f_3(13)$		$f_3(15)$		$f_3(15)$	

Stage 4 data shuffle

HB

Final Output

$f_3(0)$	$f_3(1)$	$f_3(2)$	$f_3(3)$
$f_3(4)$	$f_3(5)$	$f_3(6)$	$f_3(7)$
$f_3(8)$	$f_3(9)$	$f_3(10)$	$f_3(11)$
$f_3(12)$	$f_3(13)$	$f_3(14)$	$f_3(15)$

Stage 4 data 배열

Vertical shuffle

Perfect Shuffle(1)

◇ Perfect shuffle 방식

- perfect shuffle 이용하여 FFT 연산을 parallel하게 처리
- stage간의 입출력 양식이 같은 Flow graph로 제어를 쉽게 함

◇ Shuffle Connection

- 입력 시퀀스를 다음과 같이 나타낼 때

$$n = a_{v-1}2^{v-1} + a_{v-2}2^{v-2} + a_{v-3}2^{v-3} + \dots + a_12^1 + a_02^0, \quad N = 2^v$$

- BF 연산의 결과를 다음과 같이 연결함

$a_{v-1}=0$, Node $n \rightarrow$ Node $2n$ 에 shuffle connection

$a_{v-1}=1$, Node $n \rightarrow$ Node $(2n-N+1)$ 에 shuffle connection

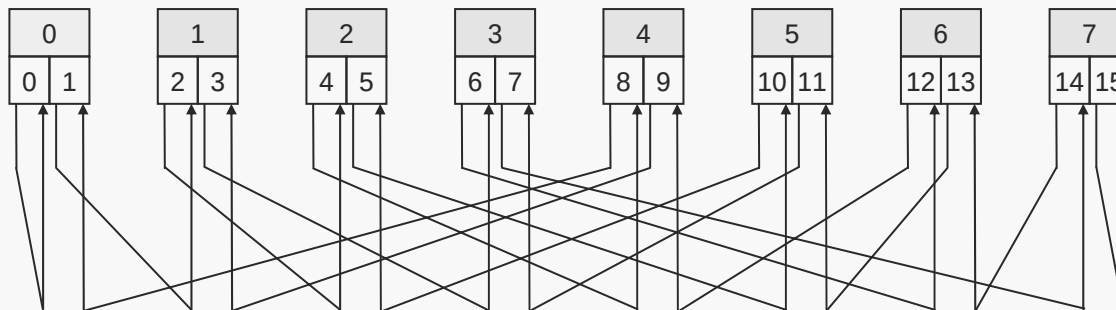


Perfect Shuffle(2)

◆ Perfect Shuffle for N=16

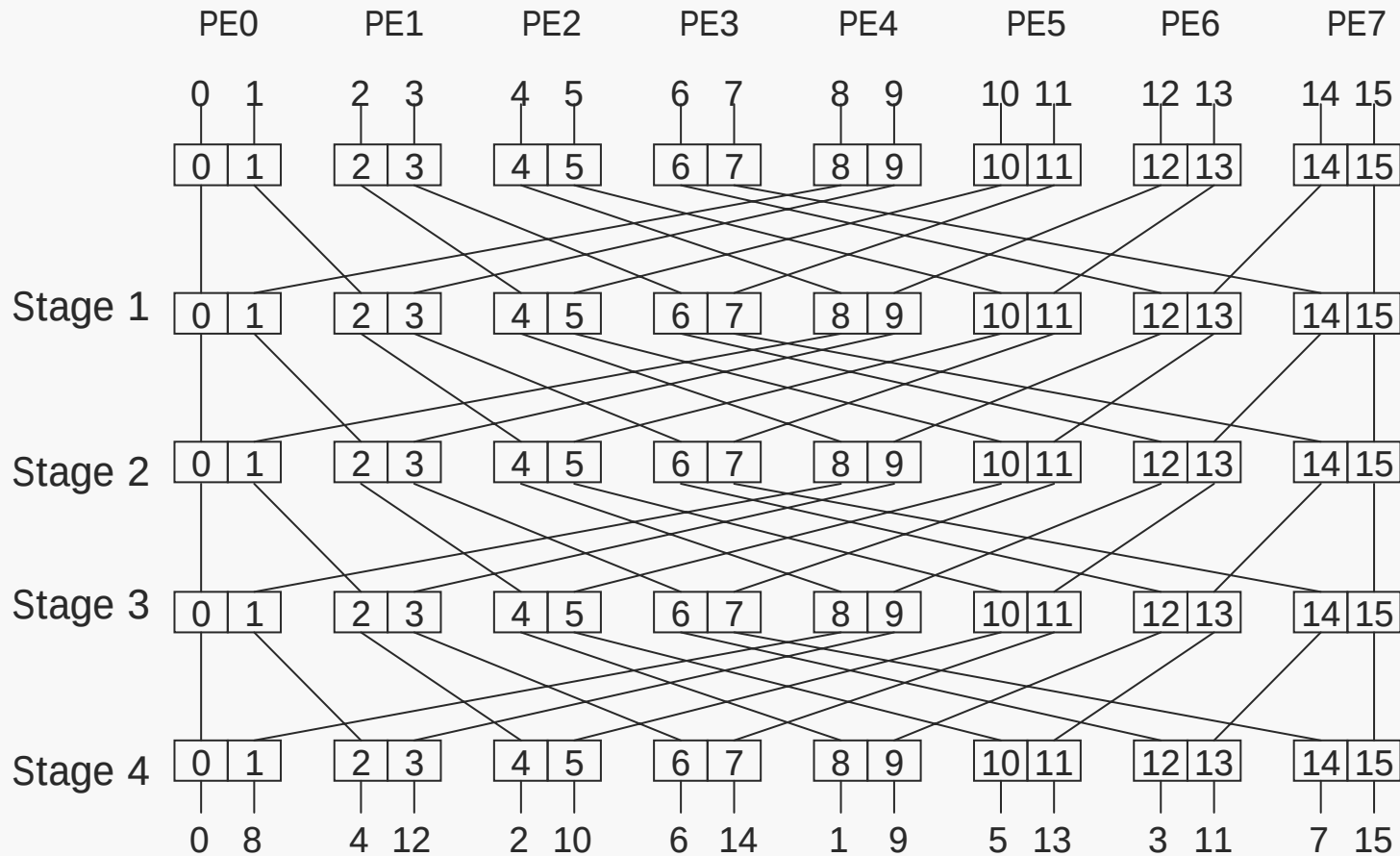
Initial value	Perfect shuffle
0000 : 0	0000 : 0
0001 : 1	0010 : 2
0010 : 2	0100 : 4
0011 : 3	0110 : 6
0100 : 4	1000 : 8
0101 : 5	1010 : 10
0110 : 6	1100 : 12
0111 : 7	1110 : 14
1000 : 8	0001 : 1
1001 : 9	0011 : 3
1010 : 10	0101 : 5
1011 : 11	0111 : 7
1100 : 12	1001 : 9
1101 : 13	1011 : 11
1110 : 14	1101 : 13
1111 : 15	1111 : 15

◆ Perfect Shuffle Graph for N=16



Perfect Shuffle 방식 (계속)

◆ Perfect shuffle를 이용한 16-point FFT 연산 과정



Comparison of Three FFT Structures

Structure		# of BFs	BF step	Dominant area factor	Time
Single processor		1	$N/2 \log_2 N$	Memory	$O(N \log_2^2 N)$
Pipelined processors*		$\log_2 N$	Pipelined	Memory	$O(N \log_2 N)$
Parallel processors	Perfect shuffle	$N/2$	$\log_2 N$	Interconnection	$O(\log_2^2 N)$
	Systolic array	N	$\log_2 N$	BFs	$O(N^{1/2})^{**}$

- ◇ $O(\cdot)$ means “grows at most as fast as”
- ◇ MAC 연산 시간을 $\log_2 N$ 이라 가정
- ◇ * : Pipelined processor는 R2SDF 방식 기준
- ◇ ** : Systolic array processor에서 처리시간은 BF보다 stage간의 data shuffle 시간이 dominant factor

